

Erasmus Mundus Joint Master Artificial Intelligence
for Sustainable Societies (AISS)

Topics on Machine Learning and its Applications

Model evaluation

2nd part

Pedro Strecht



Model evaluation (2nd part)

Decision thresholds

Performance metrics for classification

Performance metrics for regression

Multiclass evaluation

Methods to estimate a model's performance

Other evaluation criteria

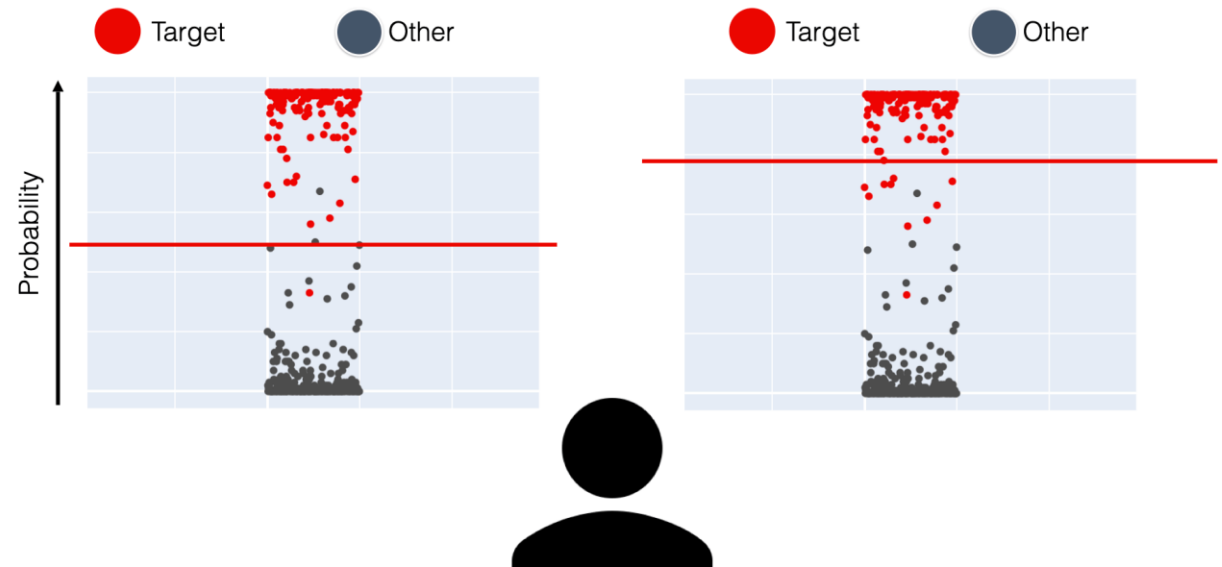
Case studies of model evaluation

General experimental set-up

Model evaluation

Decision thresholds

- Most models provide predictions as **probabilities**, not as class labels
- It is not immediately clear how to turn the probabilities into classes
- A **decision threshold** is necessary to specify the probability value above which an example is classified as **positive**

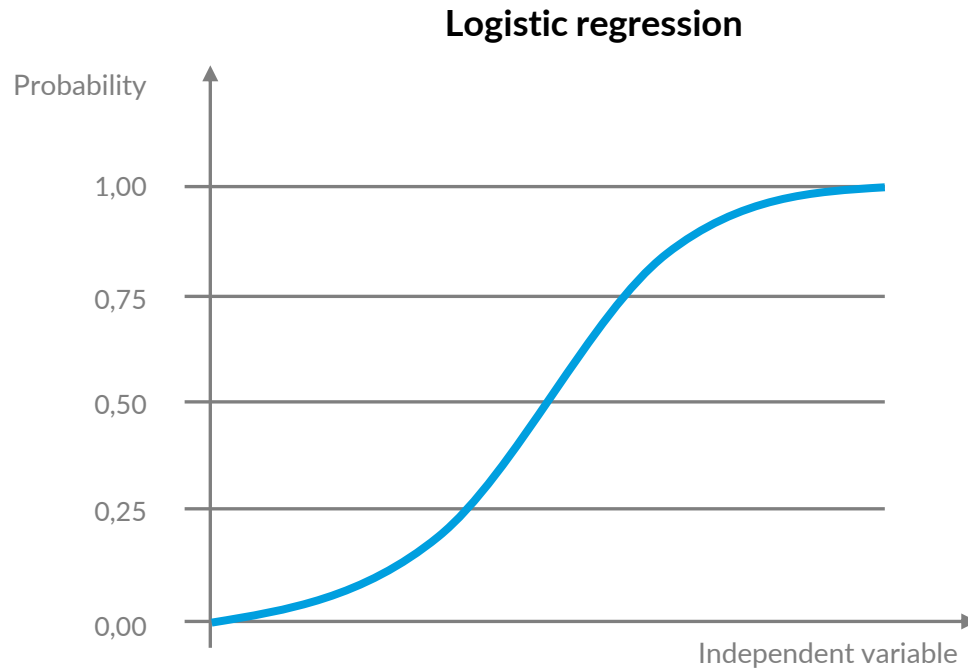


How to use classification threshold to balance precision and recall: <https://www.evidentlyai.com/classification-metrics/classification-threshold>

Model evaluation

Decision thresholds

- Model outputting probabilities



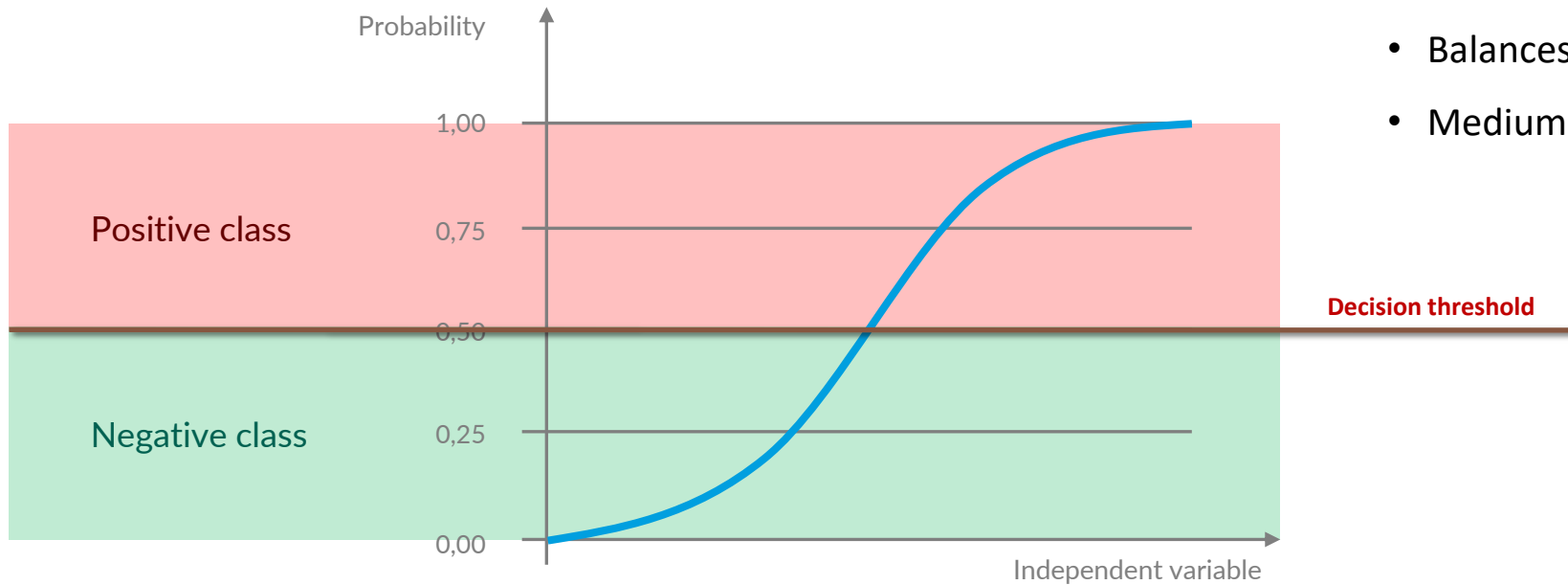
**How are examples classified
as positive or negative?**

Model evaluation

Decision thresholds

- Decision threshold = **0.50**

Logistic regression



Medium threshold

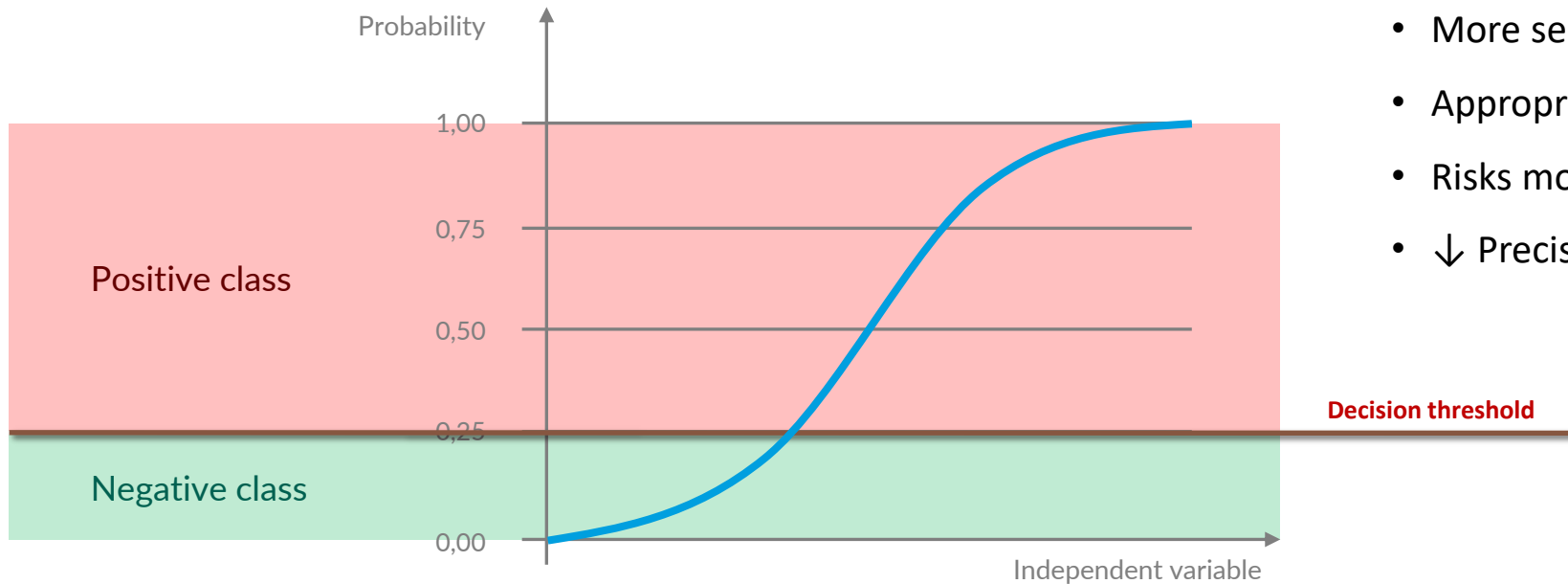
- Default choice for many models
- Balances false positives and false negatives
- Medium Precision and Recall

Model evaluation

Decision thresholds

- Decision threshold = **0.25**

Logistic regression



Lower threshold

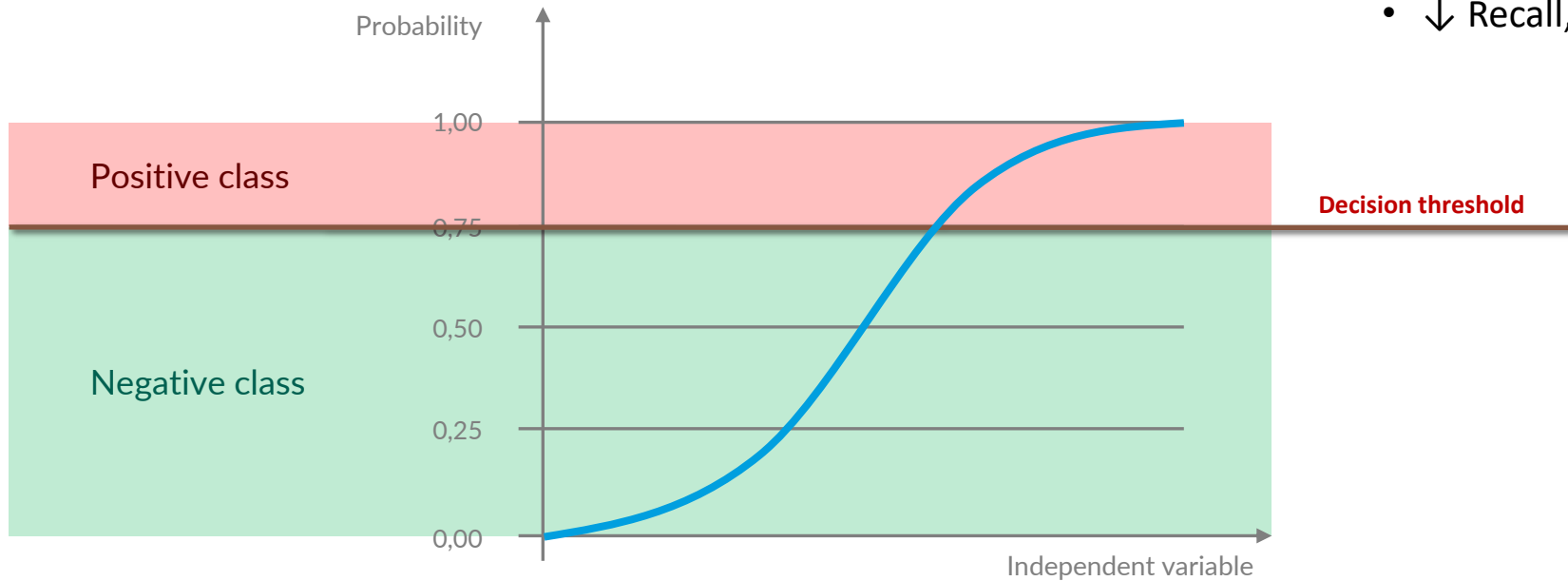
- More sensitive, fewer false negatives
- Appropriate if missing a positive case is very costly
- Risks more false positives
- ↓ Precision, ↑ Recall (sensitivity)

Model evaluation

Decision thresholds

- Decision threshold = **0.75**

Logistic regression



Higher threshold

- More specific, fewer false positives
- Appropriate if false alarms are costly
- Risks more false negatives
- ↓ Recall, ↑ Precision

Model evaluation

Decision thresholds

Example	True Label	Predicted Probability	Predicted Class (0.25)	Predicted Class (0.50)	Predicted Class (0.75)
1	1	0.85	1	1	1
2	0	0.40	1	0	0
3	1	0.65	1	1	0
4	0	0.30	1	0	0
5	1	0.55	1	1	0
6	0	0.60	1	1	0
7	1	0.90	1	1	1
8	0	0.20	0	0	0
9	1	0.75	1	1	1
10	0	0.50	1	1	0
11	1	0.35	1	0	0
12	0	0.80	1	1	1
13	1	0.60	1	1	0
14	0	0.10	0	0	0
15	1	0.95	1	1	1
16	0	0.45	1	0	0
17	1	0.70	1	1	0
18	0	0.55	1	1	0
19	1	0.80	1	1	1
20	0	0.25	1	0	0

Threshold	Precision	Recall	F1 Score	Characteristics
0.25	0.556	1.0	0.714	• Very sensitive • Catches most positives, many false positives • High Recall • Low Precision
0.50	0.692	0.9	0.789	• Balanced threshold • Good trade-off between Precision and Recall
0.75	0.833	0.5	0.625	• Very specific; fewer false positives, but misses many positives • High Precision • Low Recall

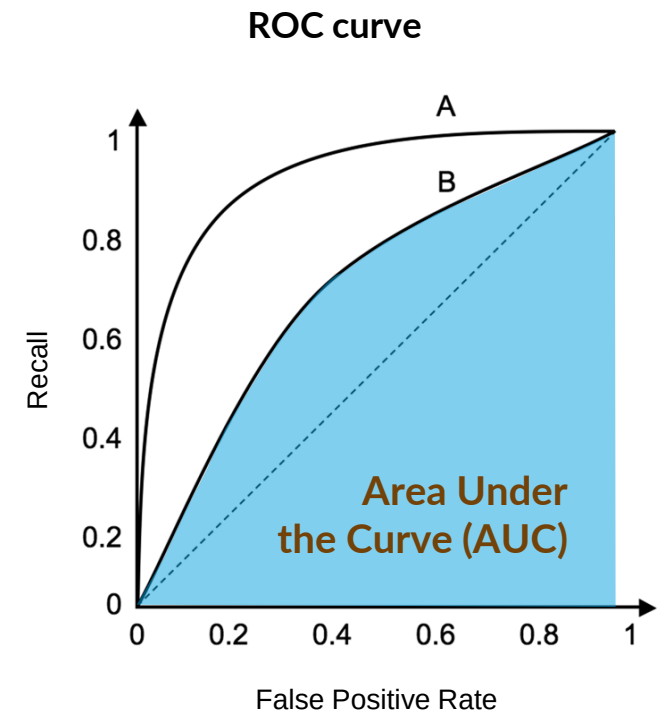
Model evaluation

Performance metrics for classification

- Metrics like F_1 Score, MCC , and Cohen's Kappa (κ) are computed **after applying a threshold**
 - Changing the threshold can **drastically change the metric**
 - Is there a metric that captures a model's ability to distinguish positives from negatives **across all possible thresholds**?

- **ROC AUC — Receiver Operating Characteristic Area Under the Curve**

- Plots curve (ROC) of **True Positive Rate (Recall)** vs **False Positive Rate**
- For all thresholds (between 0.0 and 1.0)
- Calculates the **Area Under the Curve (AUC)**



Model evaluation

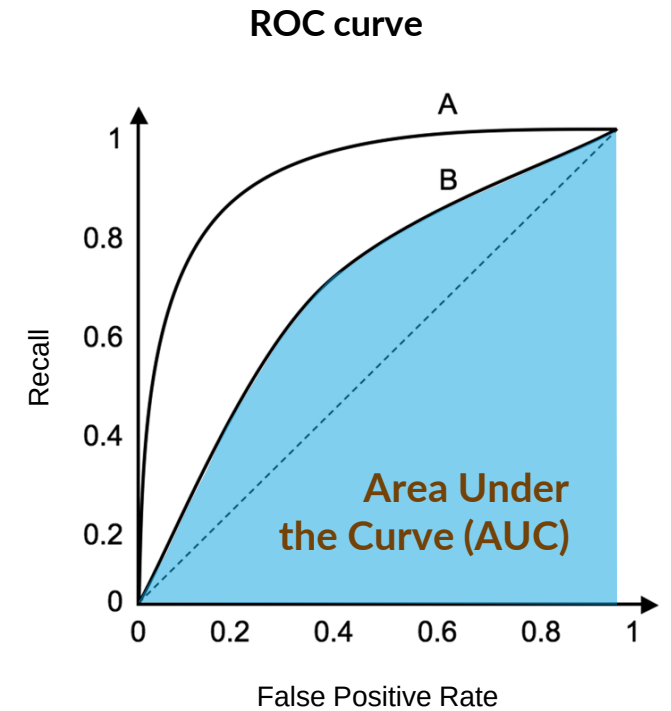
Performance metrics for classification

- **ROC AUC** captures the model's ability to distinguish positive from negative examples at all thresholds → holistic metric

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

- Ranges from 0 to 1 being interpreted as:
 - **AUC = 1.0** → perfect separation
 - **AUC = 0.5** → no discriminative power (equal to random chance)
 - **AUC = 0.0** → perfect inverse separation
- A higher **AUC** indicates better overall discrimination, regardless of the threshold chosen for classification



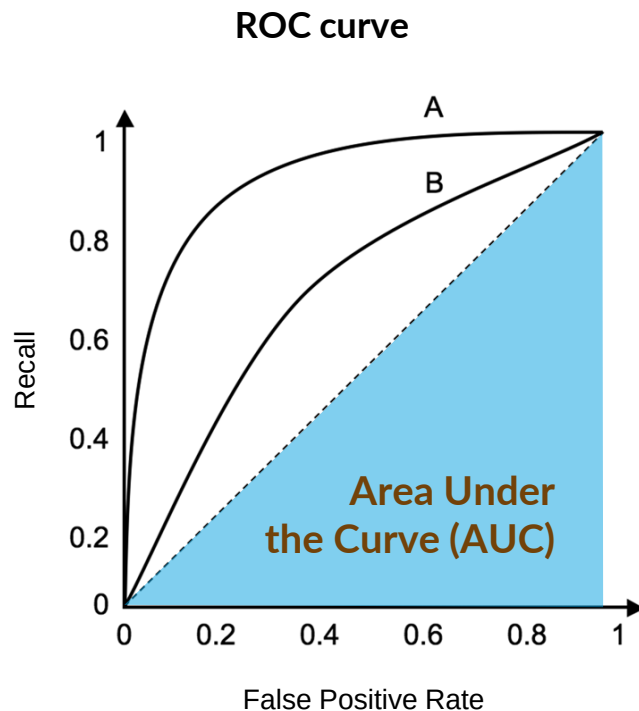
Model evaluation

Performance metrics for classification

- In essence, **ROC AUC** quantifies the model's ability to correctly rank positive and negative examples, **regardless of a specific chosen threshold**
- This characteristic makes **ROC AUC** particularly useful in imbalanced datasets, where one class significantly outnumbers the other
- In such scenarios, metrics heavily reliant on specific thresholds can be misleading, whereas **ROC AUC** provides a **more robust assessment** of the model's ability to distinguish between the classes

Model evaluation

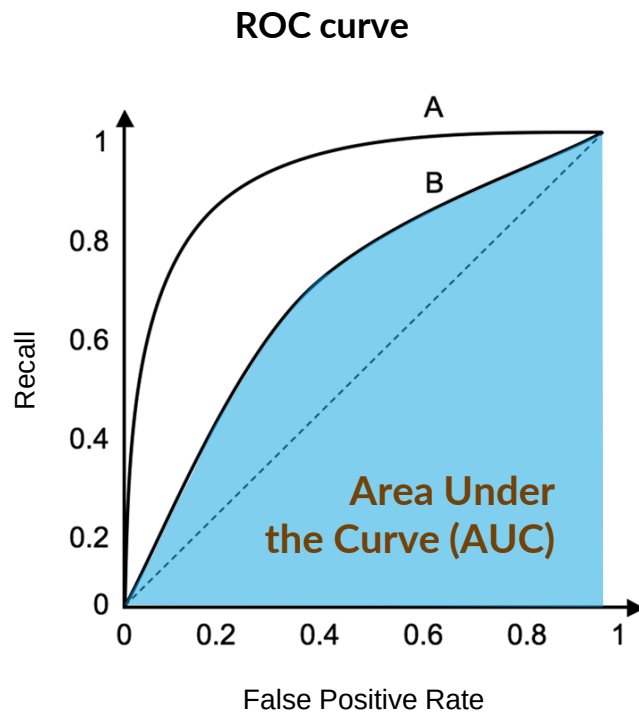
Performance metrics for classification



- Dashed line is **ROC AUC = 0.5**
- Equivalent to a model that makes random predictions
- The ROC curve coincides with the diagonal, indicating that the model has **no predictive capacity**

Model evaluation

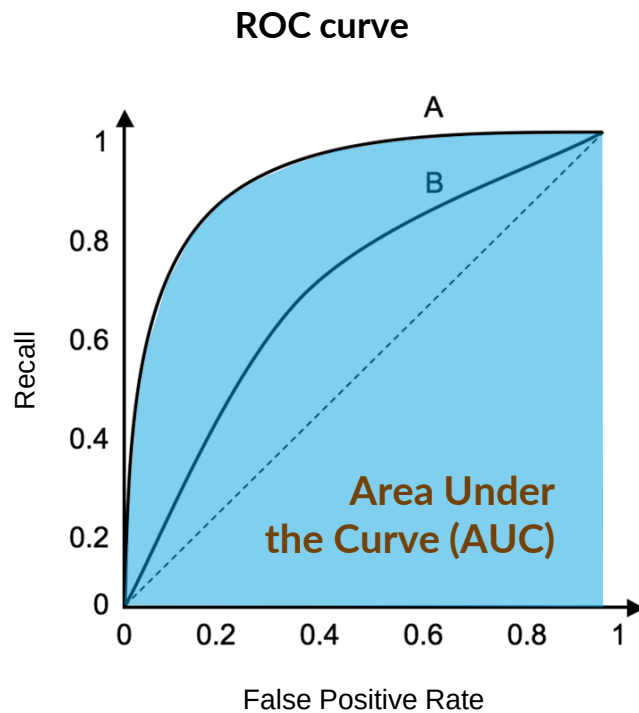
Performance metrics for classification



- Model B has **ROC AUC > 0.5**
- ROC curve deviating from the diagonal
- This demonstrates the model's ability to distinguish between the classes, **albeit with limitations**

Model evaluation

Performance metrics for classification



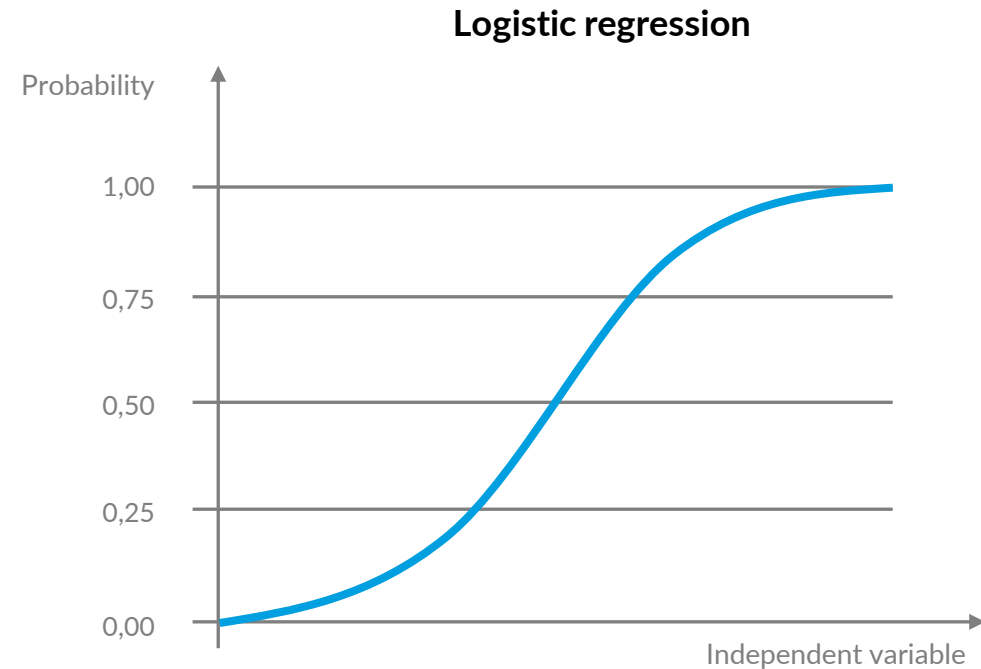
- Model A has **ROC AUC** greater than that of Model B
- ROC curve closer to the top-left corner of the graph
- This indicates **better performance in discriminating between the classes**

Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00



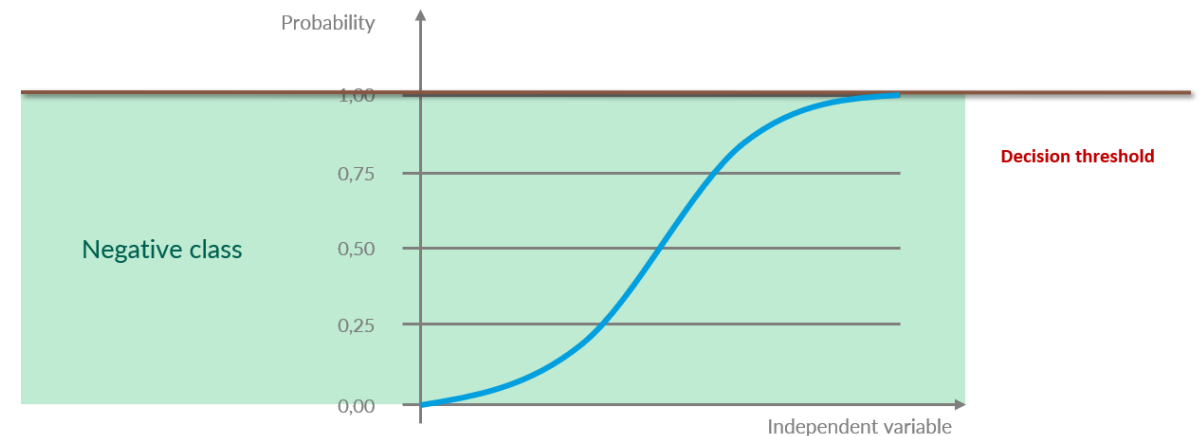
Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

All examples are predicted as **negatives** (not spam)

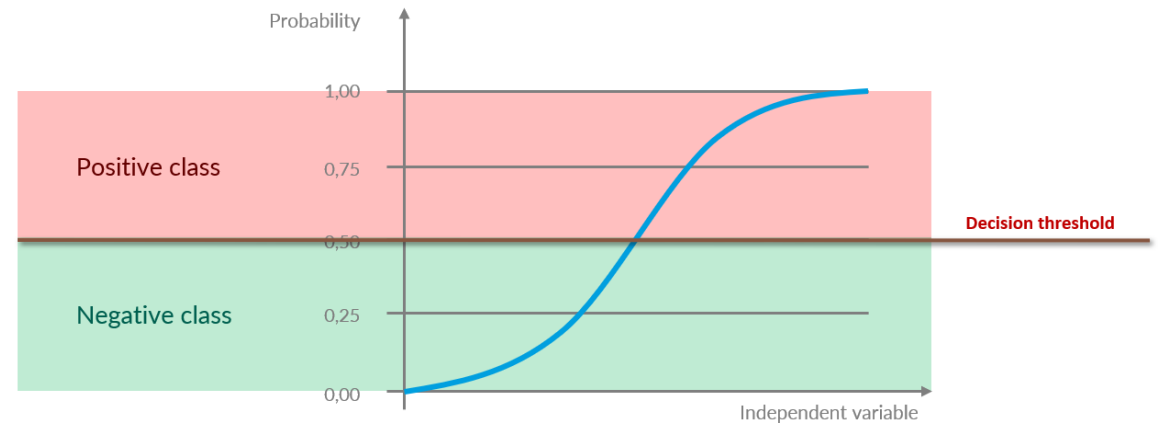


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00



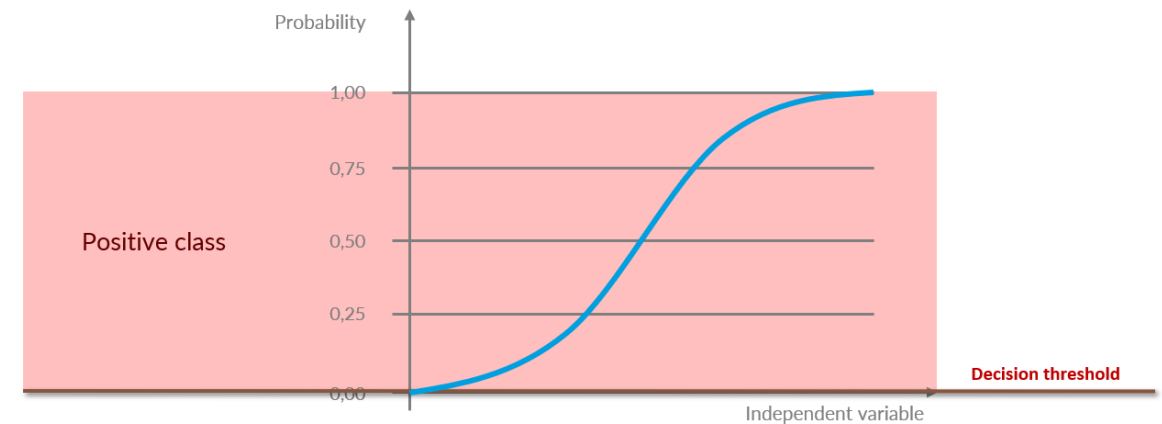
Examples are predicted as **positive** or **negative**

Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00



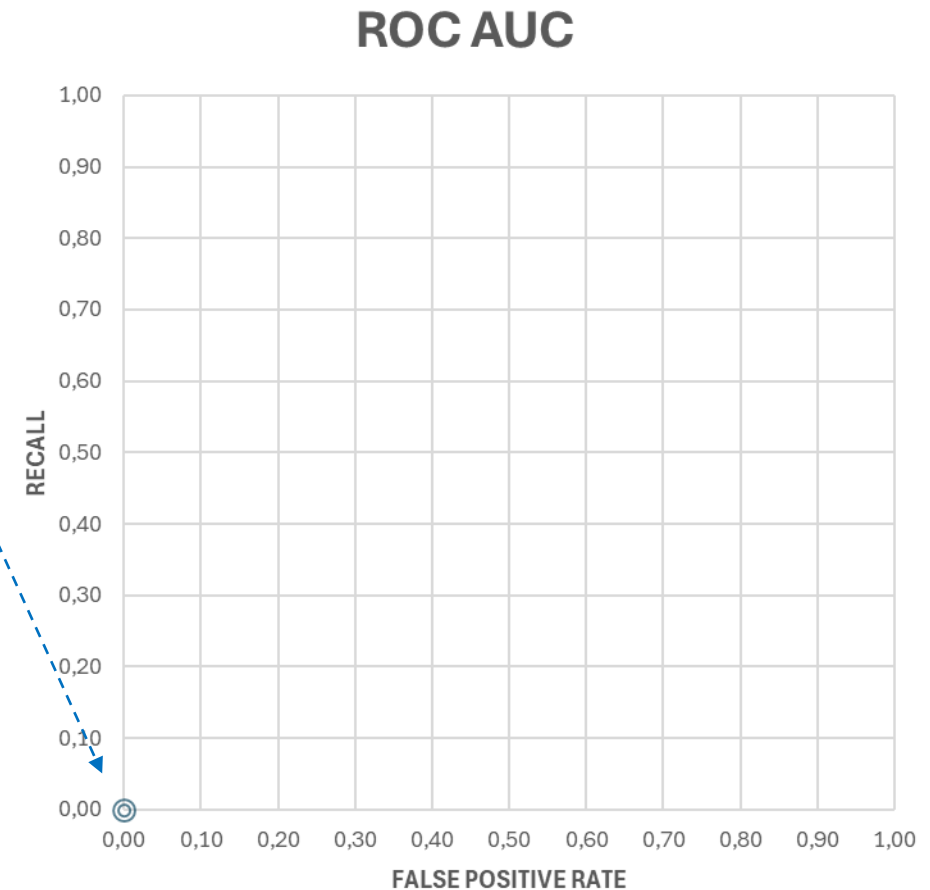
→ All examples are predicted as **positives** (spam)

Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

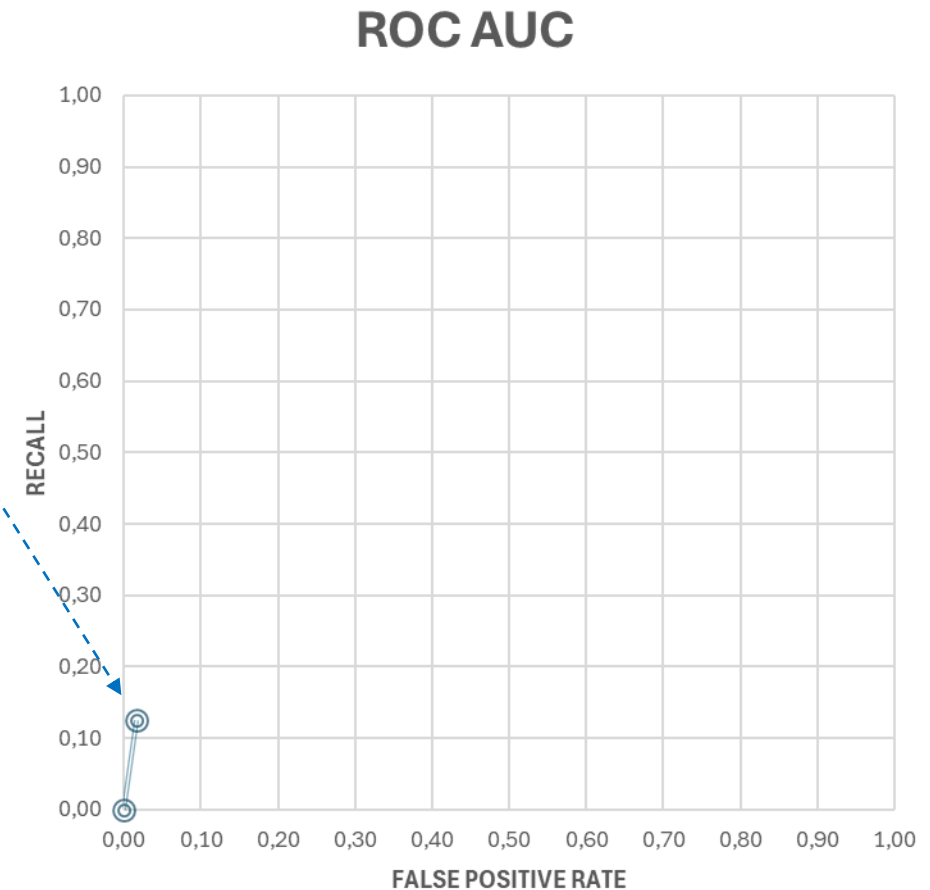


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

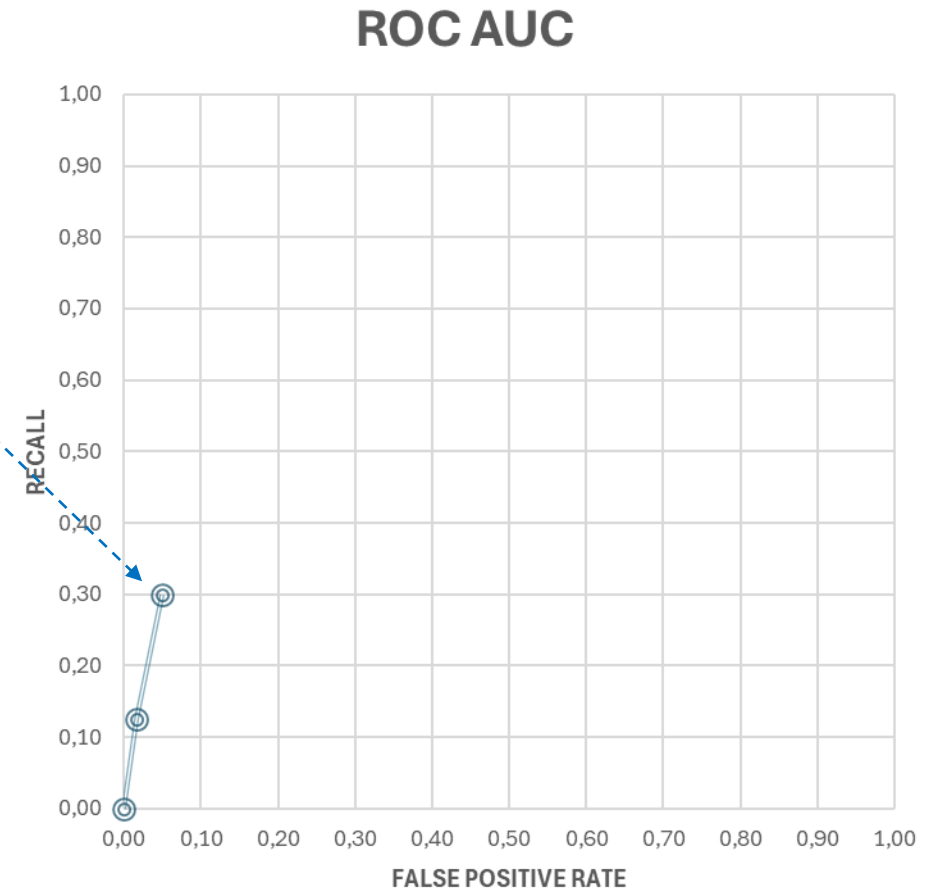


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

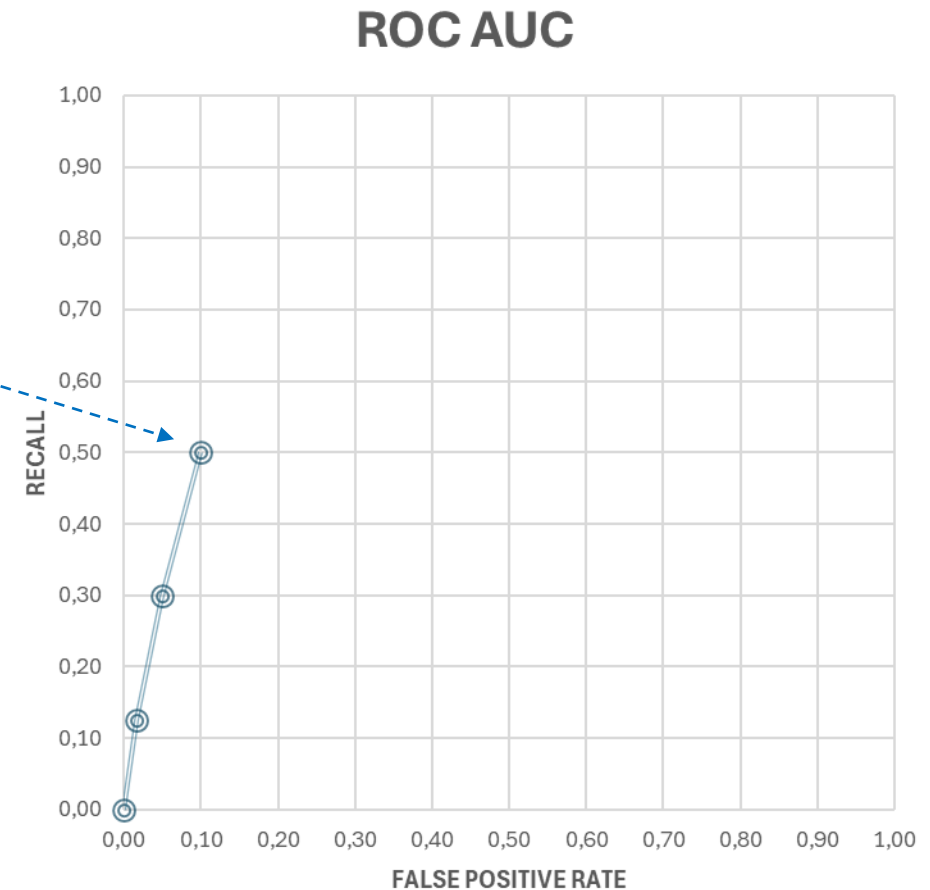


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

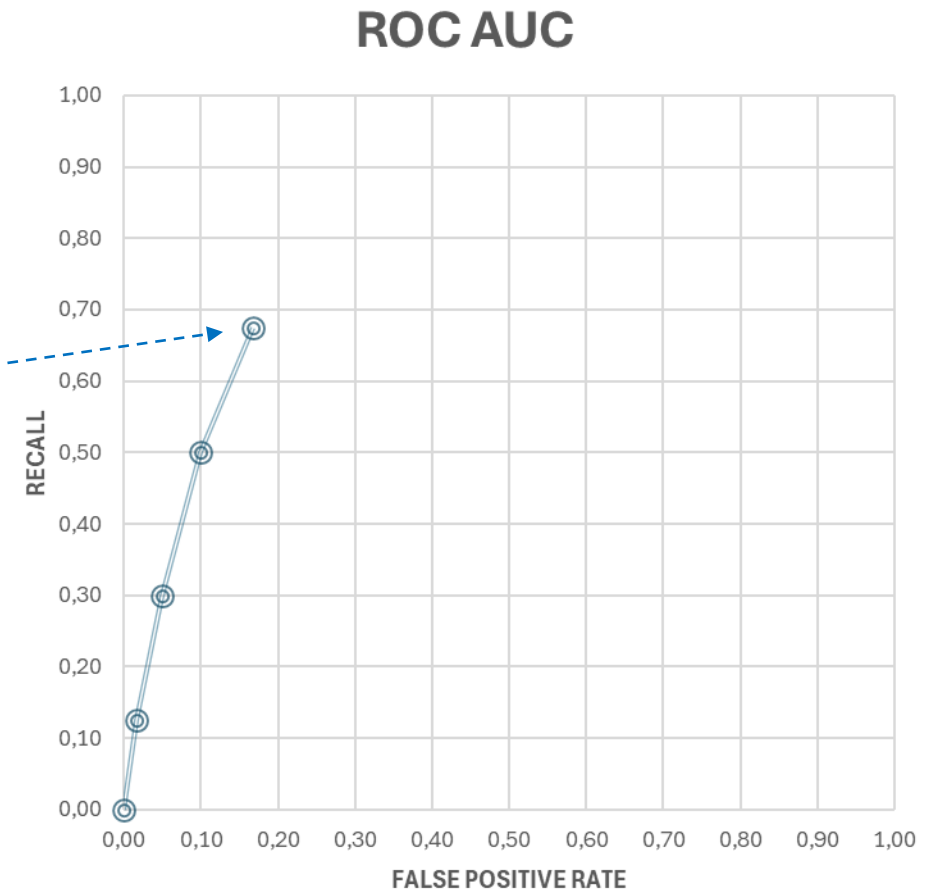


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

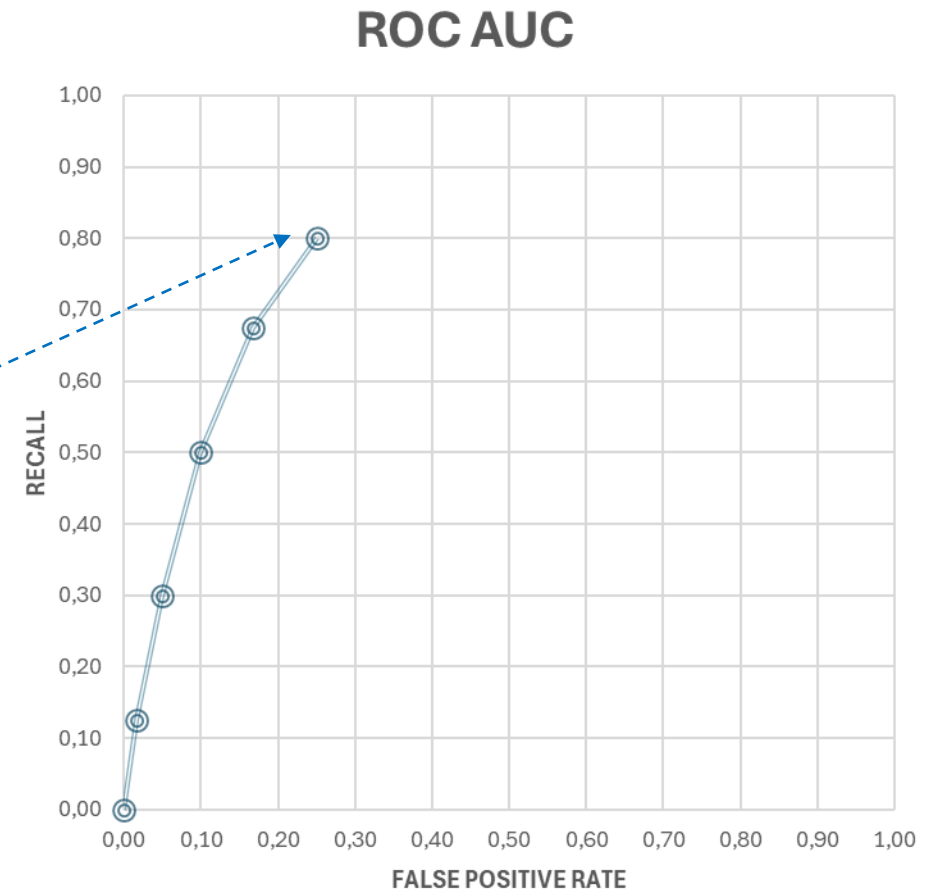


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

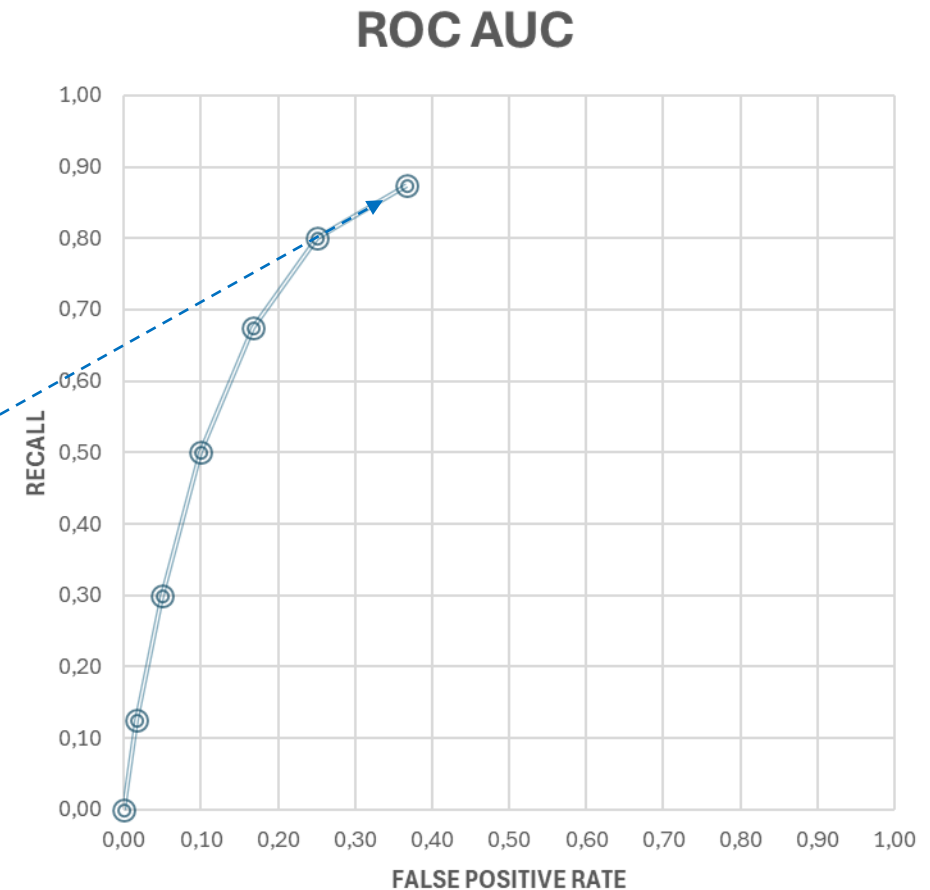


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

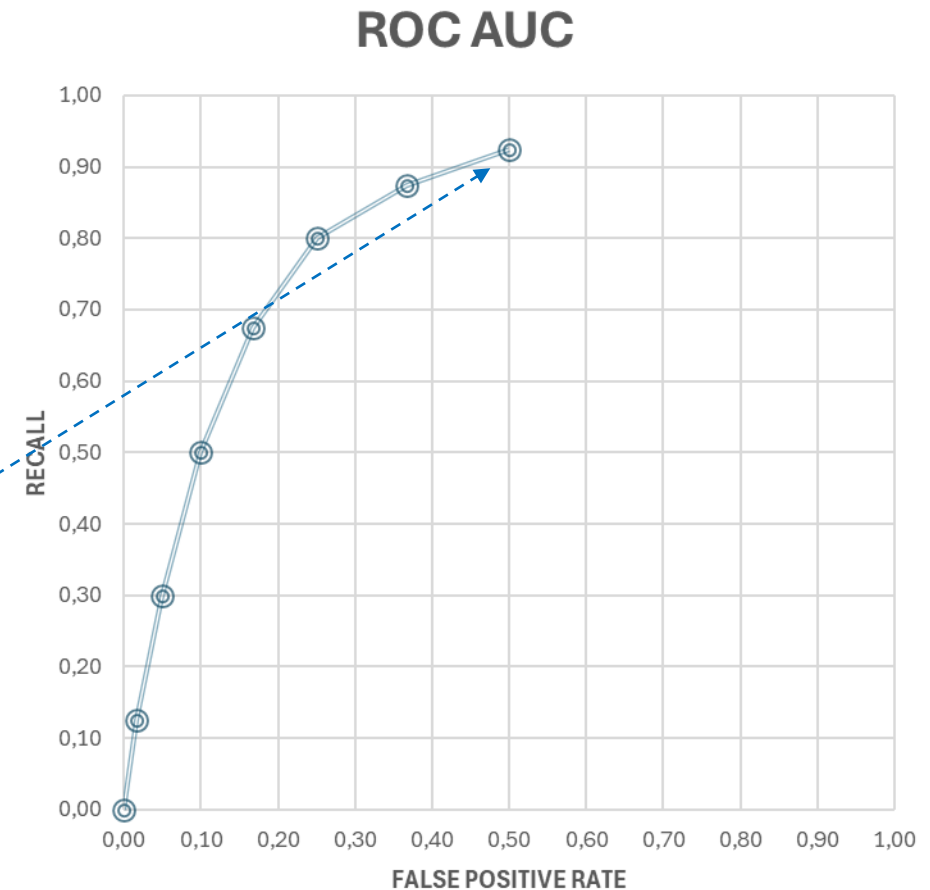


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

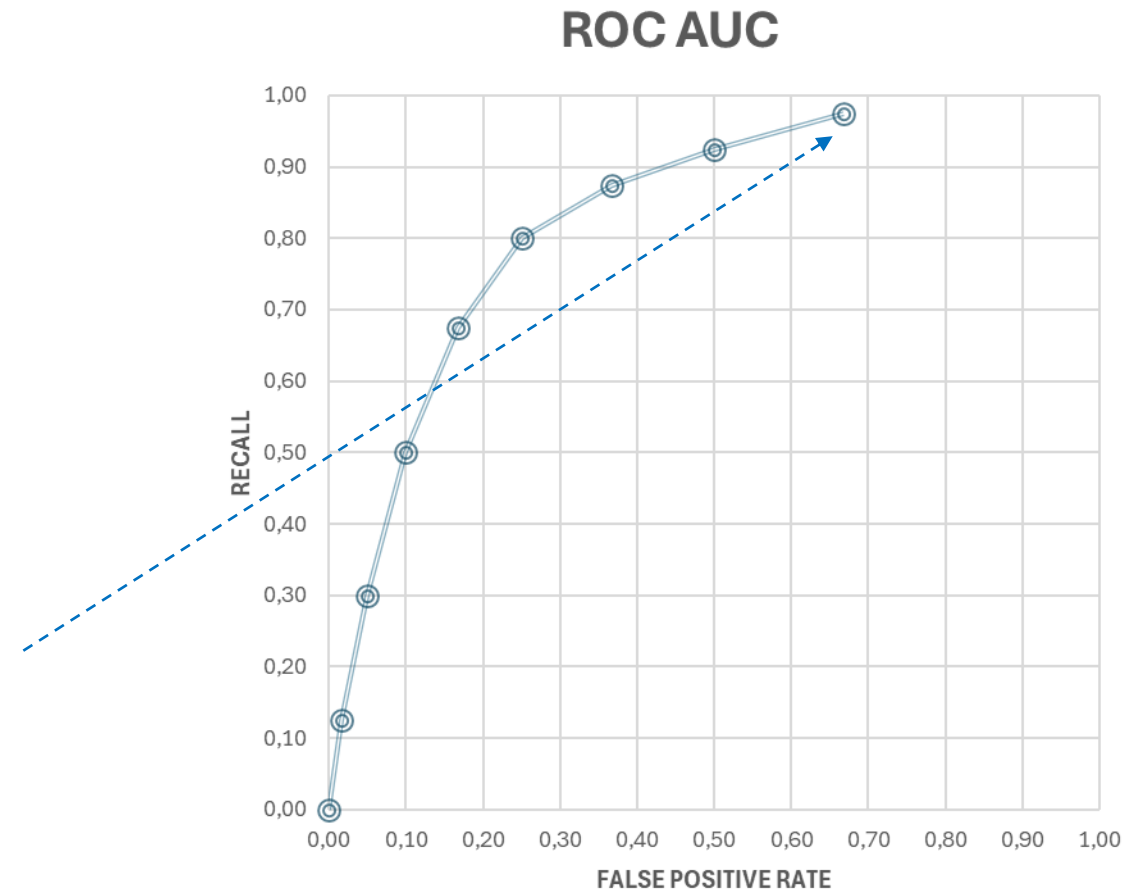


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

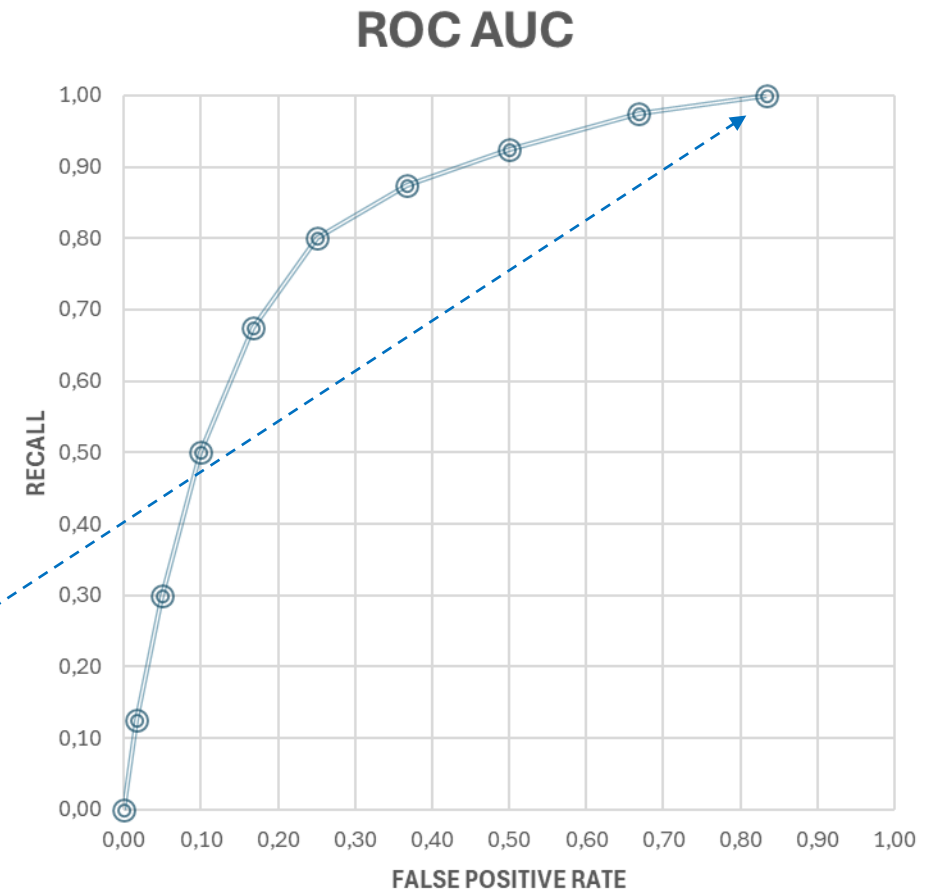


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00

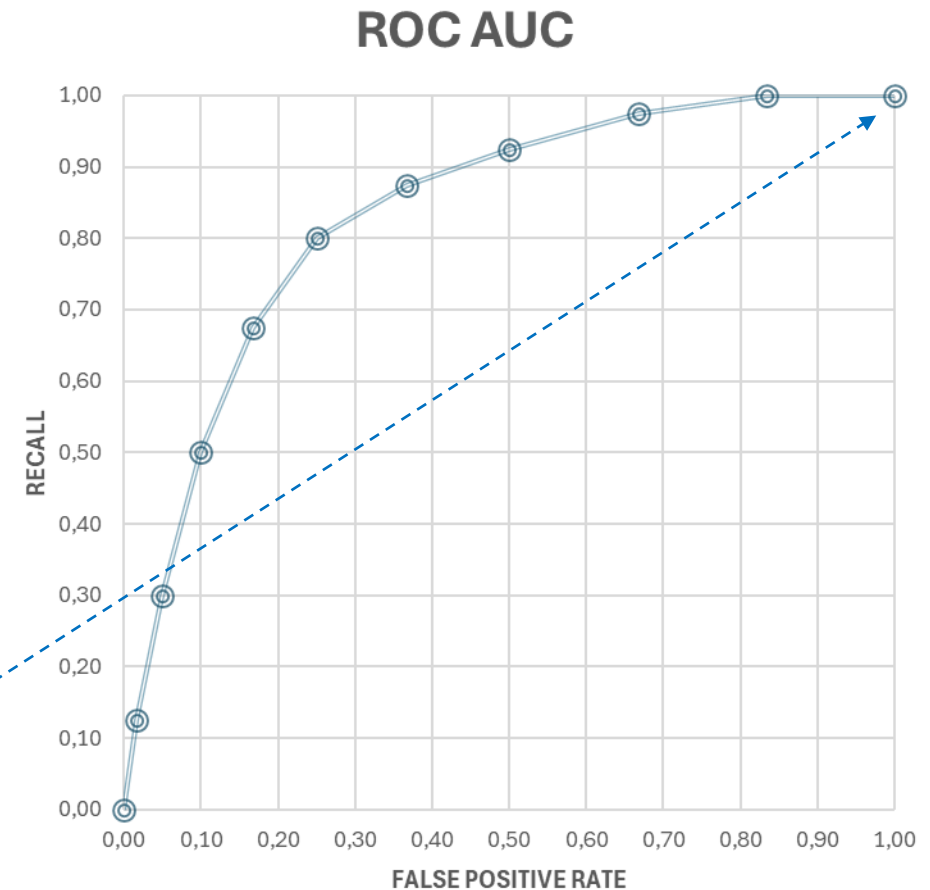


Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)

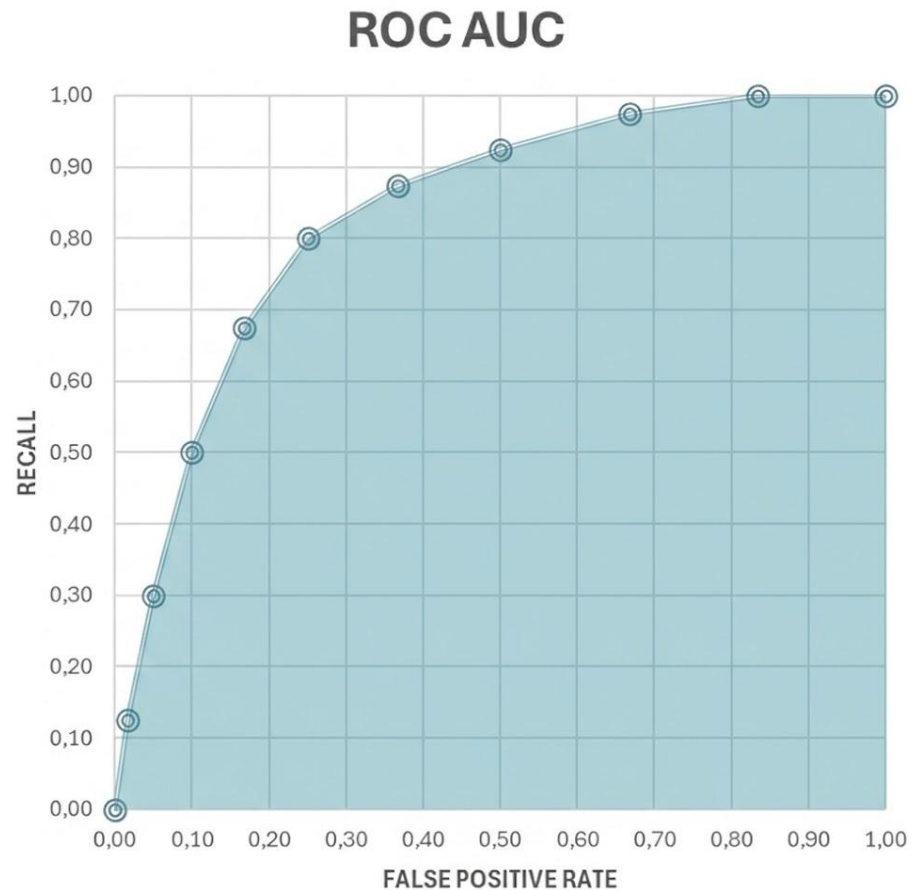
Threshold	TP	FP	TN	FN	FPR	TPR (Recall)
1,0	0	0	60	40	0,00	0,00
0,9	5	1	59	35	0,02	0,13
0,8	12	3	57	28	0,05	0,30
0,7	20	6	54	20	0,10	0,50
0,6	27	10	50	13	0,17	0,68
0,5	32	15	45	8	0,25	0,80
0,4	35	22	38	5	0,37	0,88
0,3	37	30	30	3	0,50	0,93
0,2	39	40	20	1	0,67	0,98
0,1	40	50	10	0	0,83	1,00
0,0	40	60	0	0	1,00	1,00



Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)



The *AUC* is the area under the ROC curve, therefore can be computed with an integral:

$$AUC = \int_0^1 TPR(t)FPR(t) dt$$

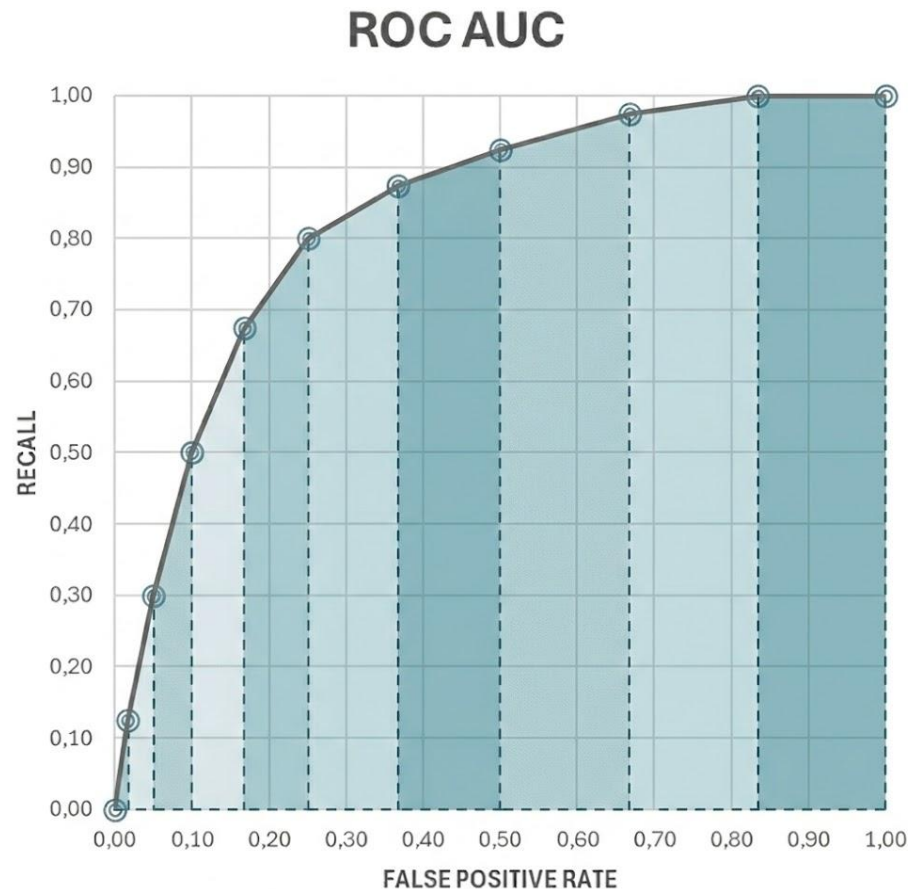
where:

t = decision threshold

Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)



The *AUC* can be approximated using the trapezoidal rule over the *ROC* points:

$$AUC = \sum_{i=0}^1 \frac{TPR_i + TPR_{i+1}}{2} (FPR_{i+1} - FPR_i)$$

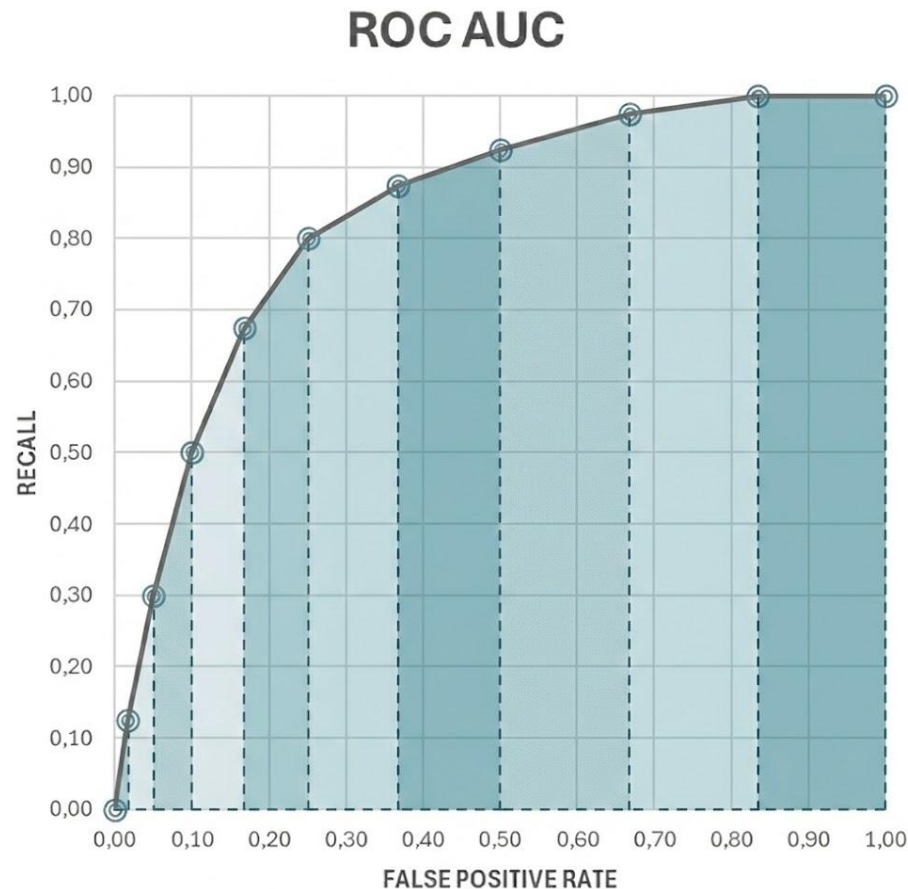
where:

i is the index of a point on the *ROC* curve

Model evaluation

Performance metrics for classification

ROC AUC computation for a model which predicts whether an e-mail is spam (1) or not spam (0)



Segment 1: $\frac{0.00+0.12}{2} \times (0.02 - 0.00) = 0.0012$

Segment 2: $\frac{0.12+0.30}{2} \times (0.05 - 0.02) = 0.0063$

Segment 3: $\frac{0.30+0.50}{2} \times (0.10 - 0.05) = 0.0200$

Segment 4: $\frac{0.50+0.67}{2} \times (0.17 - 0.10) = 0.04095$

Segment 5: $\frac{0.67+0.80}{2} \times (0.25 - 0.17) = 0.0588$

Segment 6: $\frac{0.80+0.87}{2} \times (0.37 - 0.25) = 0.1002$

Segment 7: $\frac{0.87+0.92}{2} \times (0.50 - 0.37) = 0.11635$

Segment 8: $\frac{0.92+0.97}{2} \times (0.67 - 0.50) = 0.16065$

Segment 9: $\frac{0.97+1.00}{2} \times (0.83 - 0.67) = 0.1576$

Segment 10: $\frac{1.00+1.00}{2} \times (1.00 - 0.83) = 0.1700$

$AUC \approx 0.832$

Good discriminative performance

Model evaluation

Performance metrics for regression

- **Regression** problems have their own metrics for evaluation
 - Errors (or residuals) are measured as **numerical deviations** between predicted and true values ($y_i - \hat{y}_i$)
 - Choice of metric depends on business objective and sensitivity to large errors
- Common regression metrics:
 - Mean Absolute Error (MAE)
 - Mean Squared Error (MSE)
 - Root Mean Squared Error (RMSE)
 - Mean Percentage Error (MPE)
 - Mean Absolute Percentage Error (MAPE)
 - R-squared (coefficient of determination)

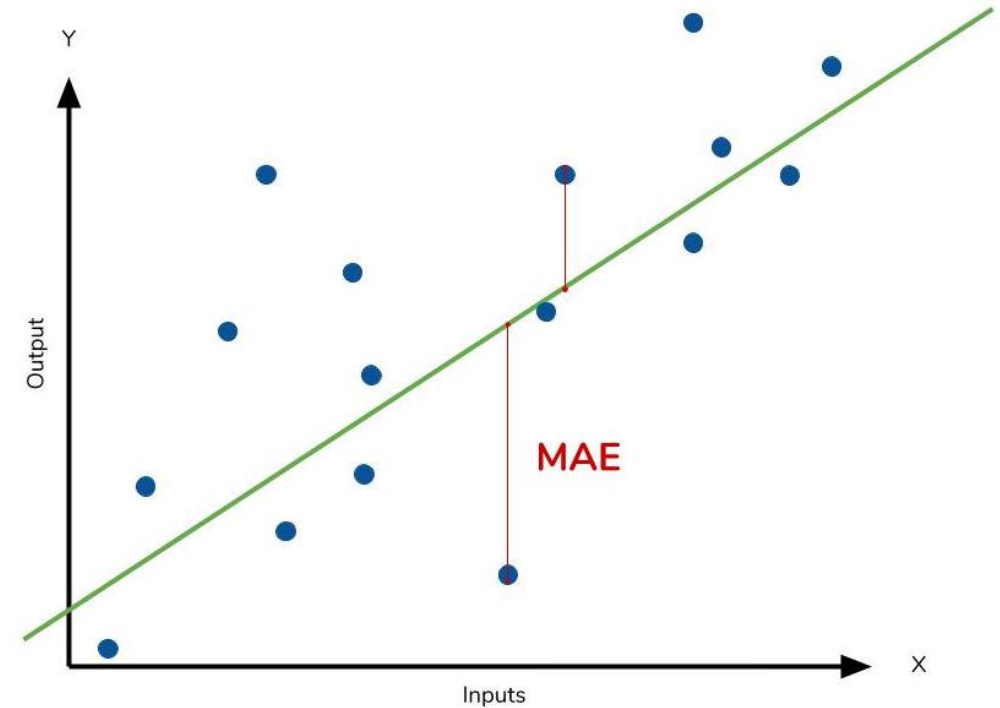
Model evaluation

Performance metrics for regression

- **Mean Absolute Error**

Measures the average magnitude of errors, regardless of whether they are positive or negative

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



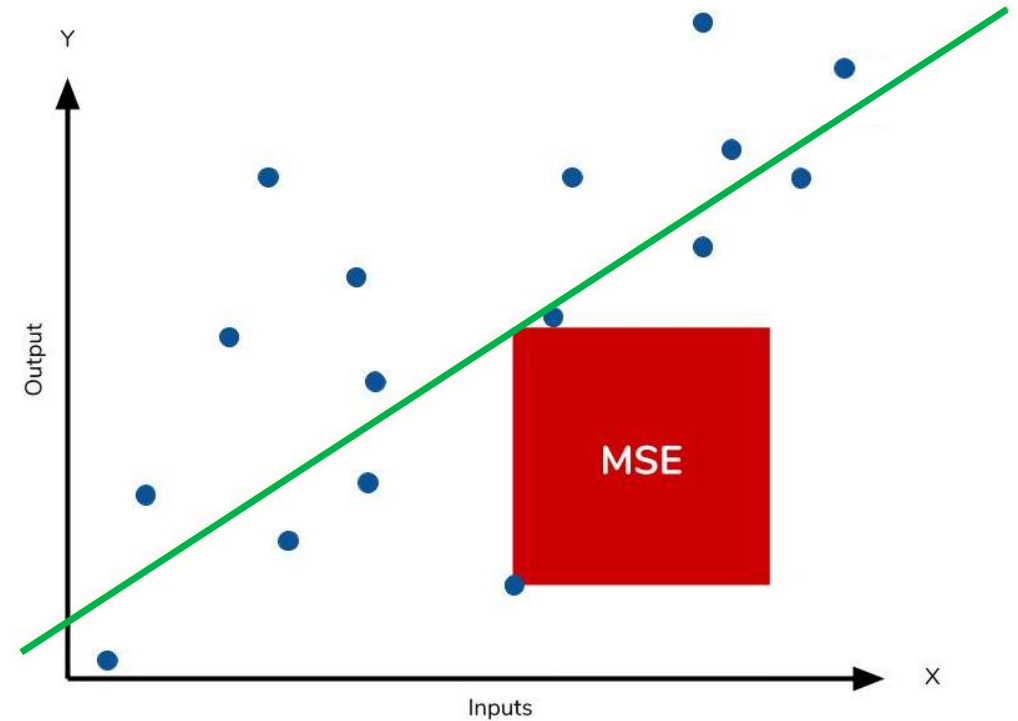
Model evaluation

Performance metrics for regression

- **Mean Squared Error**

Penalizes larger errors more heavily by squaring them, regardless of whether they are positive or negative

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



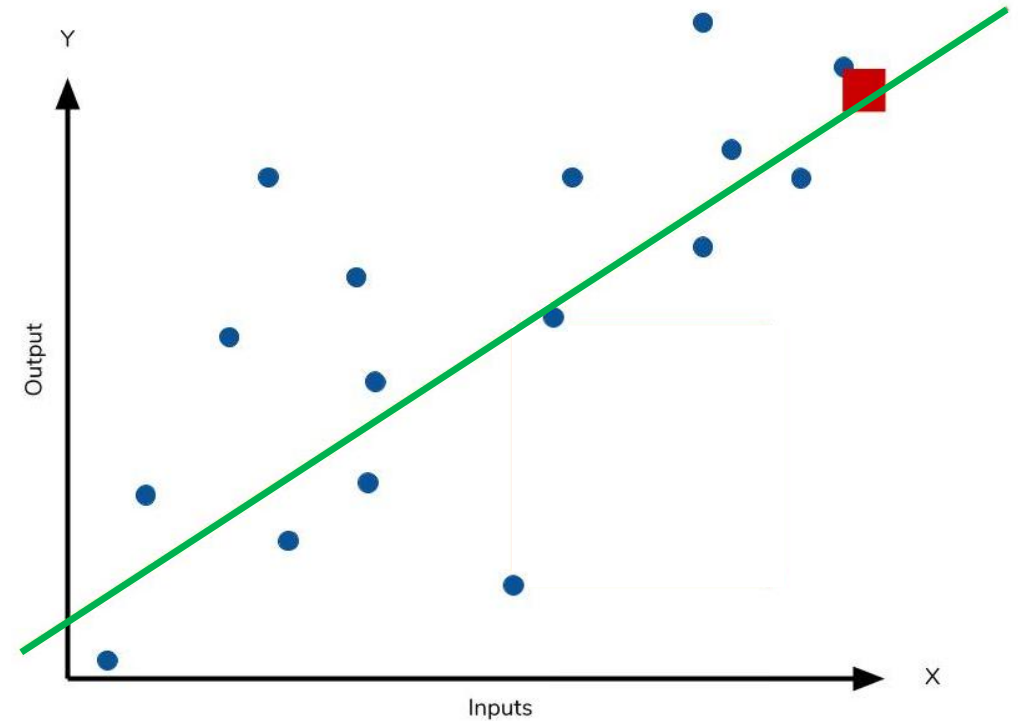
Model evaluation

Performance metrics for regression

- **Root Mean Squared Error**

Square root of MSE , bringing the units back to the original scale, regardless of whether they are positive or negative

$$RMSE = \sqrt{MSE}$$



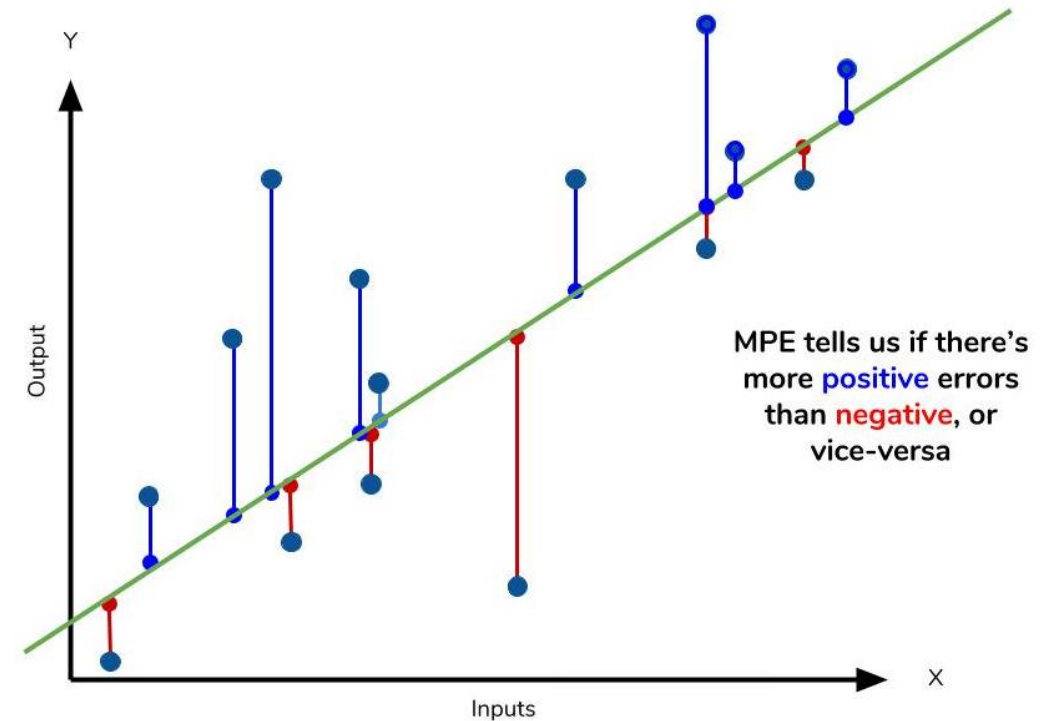
Model evaluation

Performance metrics for regression

- **Mean Percentage Error**

Measures the average magnitude of errors, considering whether they are positive or negative

$$MPE = \frac{100}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{y} \right)$$



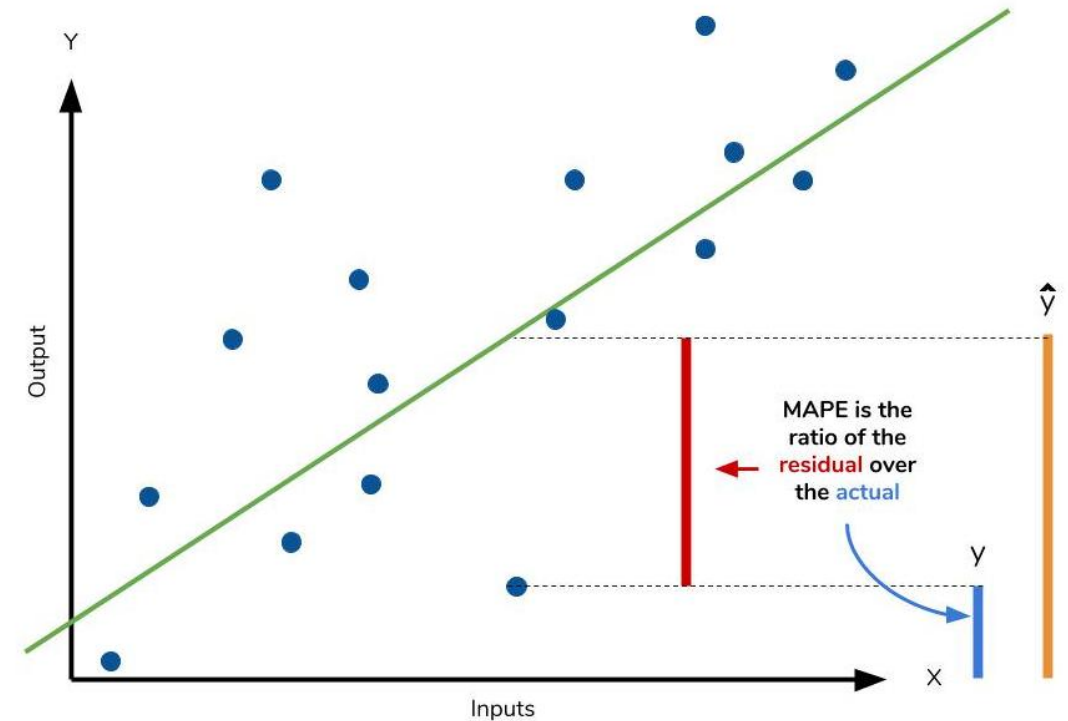
Model evaluation

Performance metrics for regression

- **Mean Absolute Percentage Error**

The percentage equivalent of MAE, expressing error as a percentage of the actual values

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y} \right|$$



Model evaluation

Performance metrics for regression

- **R-squared** (coefficient of determination)
 - Indicates how much of the variation in the target variable is explained by the model
 - It reflects how well the model's predictions match the actual data

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Formal interpretation of the result:

- $R^2 = 1 \rightarrow$ perfect fit (the model explains all variability)
- $R^2 = 0 \rightarrow$ the model explains no variability
- $R^2 < 0 \rightarrow$ the model performs worse than predicting the mean

Model evaluation

Performance metrics for regression

- **R-squared** (coefficient of determination) compares two quantities:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- **Model error (Residual Sum of Squares – RSS)** → how far the predictions are from the actual values

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Total variability (Total Sum of Squares – TSS)** → how far the actual values are from their mean

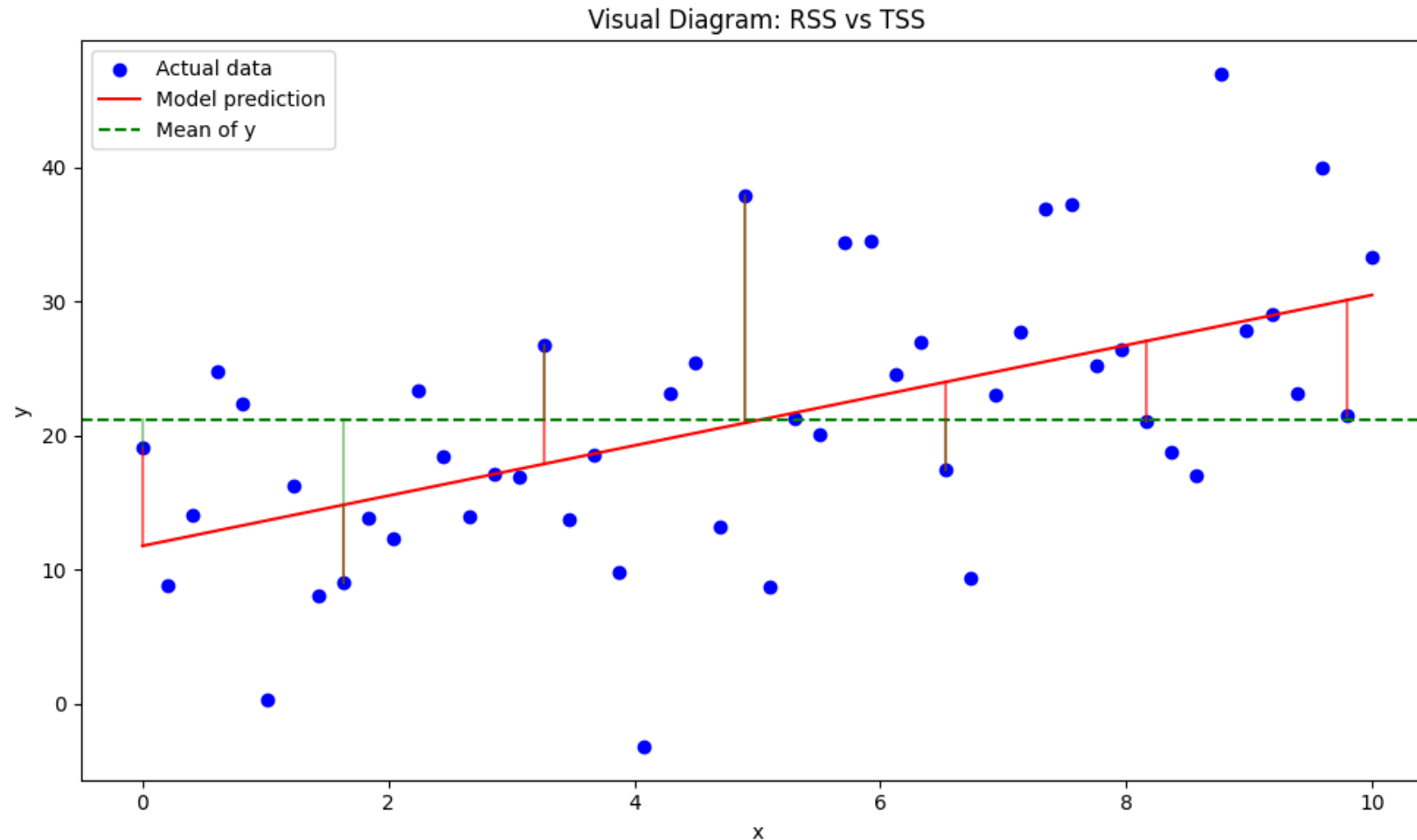
$$\sum_{i=1}^n (y_i - \bar{y})^2$$

If the model reduces the error significantly compared to the natural variability in the data, R^2 will be high

Model evaluation

Performance metrics for regression

- **R-squared** (coefficient of determination)

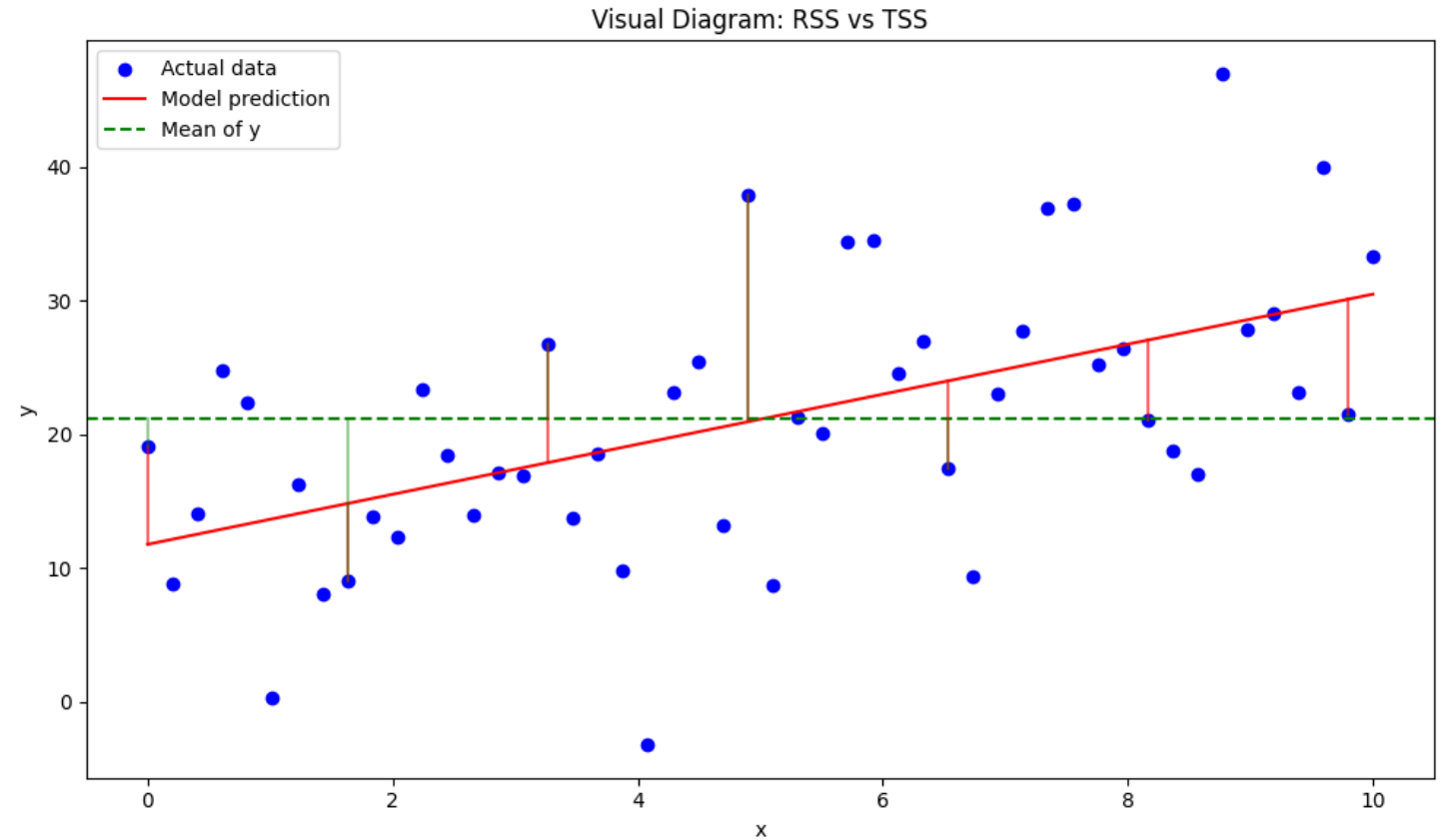


Model evaluation

Performance metrics for regression

- **Total Sum of Squares (TSS)**

$$\sum_{i=1}^n (y_i - \bar{y})^2$$



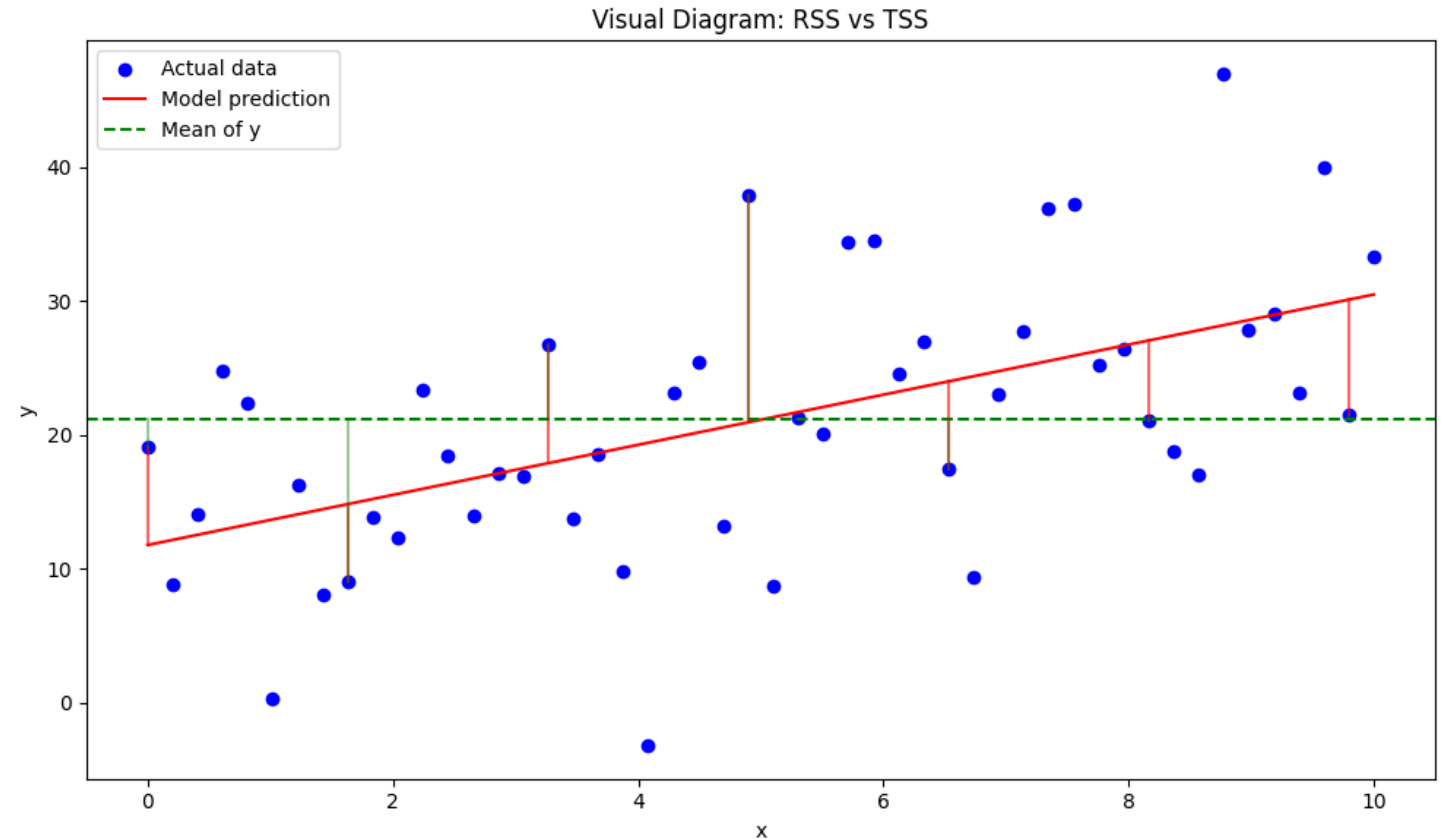
- Represents how much the **actual data points** vary around the **mean**
 - The **green** and **brown** vertical lines show how far each actual data point is from that mean
 - TSS answers the question: “How spread out is the real data, even before we fit any model?”

Model evaluation

Performance metrics for regression

- **Residual Sum of Squares (RSS)**

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

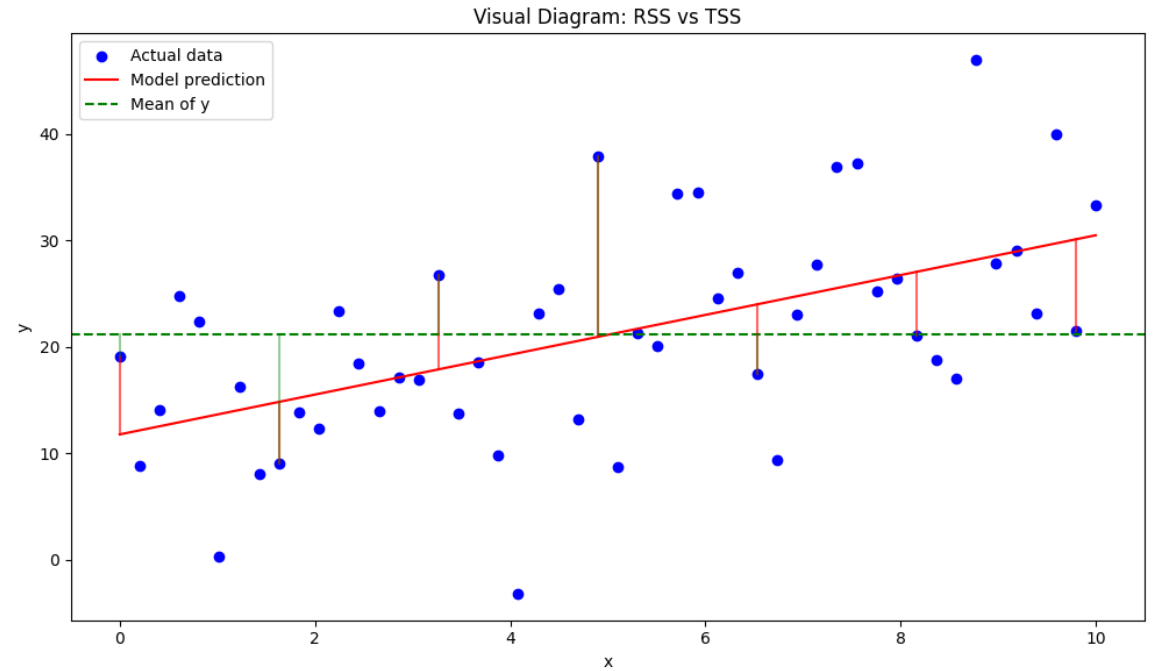


- Represents how much the **actual data points** are different from the **predictions**
 - The **red** and **brown** vertical lines show how far each actual data point is from the prediction
 - RSS answers the question: “How wrong is the model compared with the real data?”

Model evaluation

Performance metrics for regression

- **Green lines** = how far the data is from the mean
(baseline error with no model)
- **Red lines** = how far the data is from the model's prediction
(model's error)
- **Brown lines** = where the two lines overlap
- Empirical interpretation of the result:
 - If the **red lines** are much shorter than the **green lines** → the model is good (high R^2)
 - If the **red lines** are about the same size as the **green lines** → the model is useless ($R^2 \approx 0$)
 - If the **red lines** are longer than the **green lines** → the model is worse than the mean ($R^2 < 0$)



Model evaluation

Multiclass evaluation

- **Multiclass classification** refers to problems where the target variable can take on more than two classes
- Unlike binary classification, the model needs to learn how to distinguish between three or more mutually exclusive categories
 - The confusion matrix is extended for multiclass problems into an $C \times C$ **matrix**, where C is the number of classes
 - Each row represents the actual class, and each column represents the predicted class
 - **Diagonal elements** correspond to correct predictions, while off-diagonal elements show misclassifications
 - Three common methods to compute metrics are **micro-averaging**, **macro-averaging**, and **weighted macro-averaging**

Model evaluation

Multiclass evaluation

- **Micro-averaging method**

- Compute **TP**, **FP**, and **FN** for each class (each class takes turns as the positive class)
- Sum the global totals of **TP**, **FP**, and **FN** across all classes
- Compute the metric using the global totals
- Consequently, this method gives more weight to larger classes

$$Metric_{\mu} = Metric \left(\sum_{i=1}^C TP_i, \sum_{i=1}^C FP_i, \sum_{i=1}^C FN_i \right)$$

where: C = number of classes

Model evaluation

Multiclass evaluation

- **Macro-averaging method**

- Compute the metric independently for each class
- Average the class-level results
- Treat all classes equally, regardless of class frequency
- Consequently, this method is more sensitive to performance on minority classes

$$Metric_M = \frac{1}{C} \sum_{i=1}^C Metric_i$$

where: C = number of classes

Model evaluation

Multiclass evaluation

- **Weighted macro-averaging method**

- Computes the metric independently for each class
- Averages using class frequency as weights
- This method balances between micro and macro approaches

$$Metric_{WM} = \sum_{i=1}^C \frac{n_i}{N} Metric_i$$

where:

- C = number of classes
- n_i = number of examples in class i in the test set
- N = total number of examples in the test set

Model evaluation

Multiclass evaluation

Evaluation of a model with the Iris dataset



Iris Setosa



Iris Versicolor



Iris Virginica

		Predicted		
		c1 = setosa	c2 = versicolor	c3 = virginica
Actual	c1 = setosa	16	0	0
	c2 = versicolor	0	15	2
	c3 = virginica	0	3	12

- Evaluation with the metrics *Precision*, *Recall* and F_1 , using the methods:
 - micro-average
 - macro-average
 - weighted macro-average

Model evaluation

Multiclass evaluation – Micro-averaging method example

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

■ Calculation of class-specific sums

c1 as the **positive** class

c2 and **c3** as the negative class

		Predicted		
		c1 = setosa	c2 = versicolor	c3 = virginica
Actual	c1 = setosa	16	0	0
	c2 = versicolor	0	15	2
	c3 = virginica	0	3	12

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor					
c3 = virginica					
Total					

Model evaluation

Multiclass evaluation – Micro-averaging method example

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

■ Calculation of class-specific sums

c2 as the **positive class**

c1 and **c3** as the negative class

		Predicted		
		c1 = setosa	c2 = versicolor	c3 = virginica
Actual	c1 = setosa	16	0	0
	c2 = versicolor	0	15	2
	c3 = virginica	0	3	12

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica					
Total					

Model evaluation

Multiclass evaluation – Micro-averaging method example

		Predicted values		Total
		Positive	Negative	
Actual values	Positive	TP	FN	P_a
	Negative	FP	TN	N_a
Total		P_p	N_p	n

Calculation of class-specific sums

c3 as the **positive class**

c1 and **c2** as the negative class

		Predicted		
		c1 = setosa	c2 = versicolor	c3 = virginica
Actual	c1 = setosa	16	0	0
	c2 = versicolor	0	15	2
	c3 = virginica	0	3	12

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

Model evaluation

Multiclass evaluation – Micro-averaging method example

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

- **Micro-averaging** aggregates contributions of all classes by summing up across all classes and then calculates the metric
 - $Sum\ TP = \sum_{i=1}^3 TP_i = TP_1 + TP_2 + TP_3 = 16 + 15 + 12 = 43$
 - $Sum\ FP = \sum_{i=1}^3 FP_i = FP_1 + FP_2 + FP_3 = 0 + 3 + 2 = 5$
 - $Sum\ FN = \sum_{i=1}^3 FN_i = FN_1 + FN_2 + FN_3 = 0 + 2 + 3 = 5$
 - $Precision_{\mu} = \frac{SumTP}{SumTP+SumFP} = \frac{43}{43+5} = 0.896$
 - $Recall_{\mu} = \frac{SumTP}{SumTP+SumFN} = \frac{43}{43+5} = 0.896$
 - $F_{1\mu} = 2 \frac{Precision_{\mu} \times Recall_{\mu}}{Precision_{\mu} + Recall_{\mu}} = 0.896$

Model evaluation

Multiclass evaluation – Macro-averaging method example

- **Macro-averaging** calculates metrics for each class independently and then averages them, treating all classes equally, regardless of their size

- $Precision_i = \frac{TP_i}{TP_i + FP_i}$

- $Recall_i = \frac{TP_i}{TP_i + FN_i}$

- $F1_i = 2 \frac{Precision_i Recall_i}{Precision_i + Recall_i}$

- $Precision_M = \frac{1}{C} \sum_{i=1}^C Precision_i$

- $Recall_M = \frac{1}{C} \sum_{i=1}^C Recall_i$

- $F1_M = \frac{1}{C} \sum_{i=1}^C F1_i$

Model evaluation

Multiclass evaluation – Macro-averaging method example

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

- **Macro-averaging** calculates metrics for each class independently and then averages them, treating all classes equally, regardless of their size

- $Precision_1 = \frac{TP_1}{TP_1 + FP_1} = \frac{16}{16 + 0} = 1.0$

- $Recall_1 = \frac{TP_1}{TP_1 + FN_1} = \frac{16}{16 + 0} = 1.0$

- $F_{1_1} = 2 \frac{Precision_1 Recall_1}{Precision_1 + Recall_1} = 1.0$

c1 as the **positive** class

c2 and **c3** as the negative class

Model evaluation

Multiclass evaluation – Macro-averaging method example

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

- **Macro-averaging** calculates metrics for each class independently and then averages them, treating all classes equally, regardless of their size

- $Precision_2 = \frac{TP_2}{TP_2 + FP_2} = \frac{15}{15 + 3} = 0.833$

- $Recall_2 = \frac{TP_2}{TP_2 + FN_2} = \frac{15}{15 + 2} = 0.882$

- $F_{1_2} = 2 \frac{Precision_2 Recall_2}{Precision_2 + Recall_2} = 0.857$

c2 as the **positive class**

c1 and **c3** as the negative class

Model evaluation

Multiclass evaluation – Macro-averaging method example

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

- **Macro-averaging** calculates metrics for each class independently and then averages them, treating all classes equally, regardless of their size

- $Precision_3 = \frac{TP_3}{TP_3 + FP_3} = \frac{12}{12 + 2} = 0.857$

- $Recall_3 = \frac{TP_3}{TP_3 + FN_3} = \frac{12}{12 + 5} = 0.8$

- $F_{13} = 2 \frac{Precision_3 Recall_3}{Precision_3 + Recall_3} = 0.828$

c3 as the **positive class**

c1 and **c2** as the negative class

Model evaluation

Multiclass evaluation – Macro-averaging method example

Class	TP	TN	FP	FN	Total
c1 = setosa	16	32	0	0	48
c2 = versicolor	15	28	3	2	48
c3 = virginica	12	31	2	3	48
Total	43	91	5	5	144

- **Macro-averaging** calculates metrics for each class independently and then averages them, treating all classes equally, regardless of their size

$$\square \text{Precision}_M = \frac{1}{C} \sum_{i=1}^C \text{Precision}_i = \frac{\text{Precision}_1 + \text{Precision}_2 + \text{Precision}_3}{3} = \frac{1 + 0.833 + 0.857}{3} = 0.897$$

$$\square \text{Recall}_M = \frac{1}{C} \sum_{i=1}^C \text{Recall}_i = \frac{\text{Recall}_1 + \text{Recall}_2 + \text{Recall}_3}{3} = \frac{1 + 0.882 + 0.8}{3} = 0.894$$

$$\square F_{1M} = \frac{1}{C} \sum_{i=1}^C F1_i = \frac{F1_1 + F1_2 + F1_3}{3} = \frac{1 + 0.857 + 0.828}{3} = 0.895$$

Model evaluation

Multiclass evaluation – Weighted macro-averaging method example

- **Weighted Macro-Average** – Adjusts for class imbalance by weighting each class's metric by its number of examples

$$Precision_i = \frac{TP_i}{TP_i + FP_i} \qquad Recall_i = \frac{TP_i}{TP_i + FN_i}$$

- $Precision_{WM} = \sum_{i=1}^C \frac{n_i}{N} Precision_i$
 - $Recall_{WM} = \sum_{i=1}^C \frac{n_i}{N} Recall_i$
 - $F1_{WM} = \sum_{i=1}^C \frac{n_i}{N} F1_i$
- The **Iris** dataset is perfectly balanced so the weighted macro-averaged has the same values as macro-averaged

Model evaluation

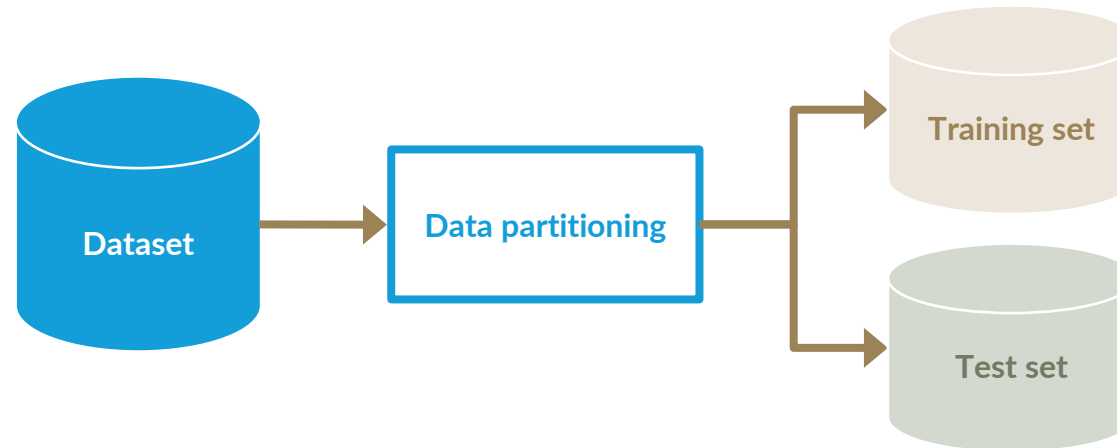
Multiclass evaluation

Averaging method	Computation	Class weighting	Effect of class imbalance
Micro-averaging	Aggregate all instances across classes	Implicit, by instance counts	Dominated by larger classes
Macro-averaging	Compute metric per class, then average	All classes equally weighted	Small classes matter as much as large ones
Weighted macro-averaging	Compute metric per class, then average weighted by class size	Weighted by number of instances	Balances class importance, accounts for imbalance

Model evaluation

Methods to estimate a model's performance

- How to arrange examples for **training** and **evaluation**?



- Two main methods:
 - Holdout
 - k -fold cross validation

Model evaluation

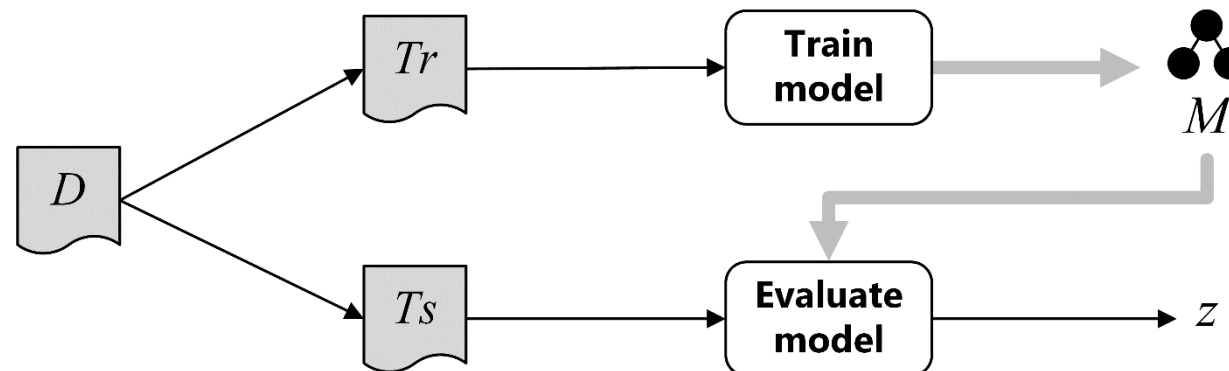
Methods to estimate a model's performance – Holdout

- The **holdout** method is a simple approach to estimating the performance of machine learning models
 - The dataset is deliberately partitioned into **two distinct subsets**
 - The training set typically comprises two-thirds of the data
 - The test set encompassing the remaining one third of the data
 - This approach is commonly considered **pessimistic** as it uses only a portion of the available data for training
 - Nevertheless, its straightforwardness and ease of application have led to its widespread adoption

Model evaluation

Methods to estimate a model's performance – Holdout

- In **holdout** data split, a dataset D is divided into two mutually exclusive partitions:
 - Training set (Tr), used to train the model M
 - Test set (Ts), used to evaluate it
- The evaluation can use any metric, such as *Accuracy*, *Precision*, *Recall*, F_1 , or others, with the result denoted by z



Model evaluation

Methods to estimate a model's performance – Holdout

- An extension of the holdout method, known as random subsampling, also referred to as **Monte Carlo cross-validation**, addresses the potential limitation of using a single training-test split
 - This method involves **repeating** the holdout process multiple times (k iterations)
 - In each iteration, a new training and test set are randomly sampled from the original dataset, following the same two-thirds to one-third split ratio
 - The model is then trained on each unique training set and evaluated on its corresponding test set
 - The overall performance of the model is then estimated by **averaging the performance** metrics obtained across all k iterations
 - This approach helps to mitigate the potential bias introduced by a single random split and provides a more robust estimate of the model's generalizability

Model evaluation

Methods to estimate a model's performance – k -fold cross-validation

- The **k -fold cross-validation** method is a robust approach for evaluating the performance of machine learning models
- In this method, the dataset is randomly partitioned into k disjoint (nonoverlapping) subsets of roughly equal size, known as **folds**

$$D = f_1 \cup \dots \cup f_k$$

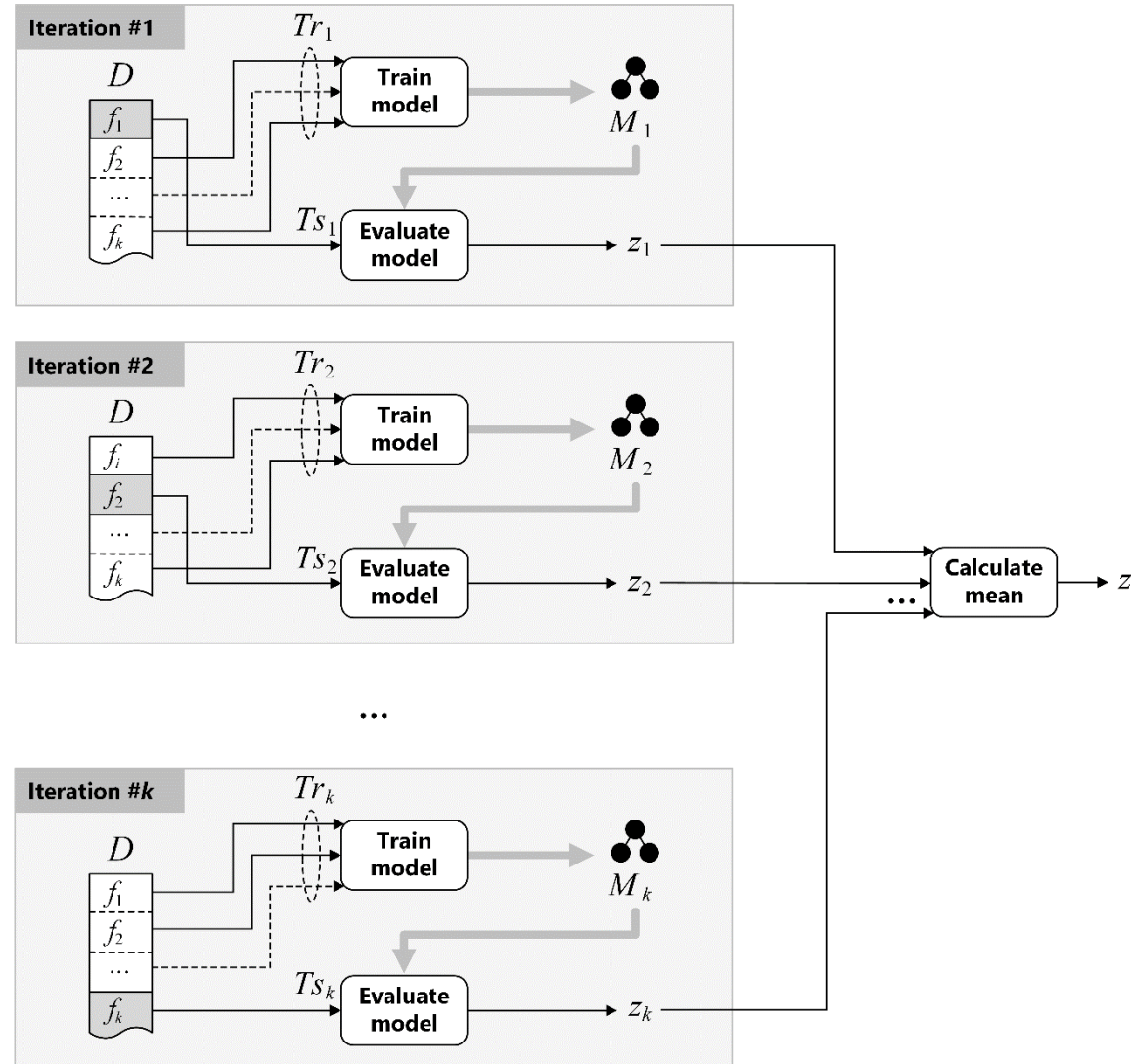
- Each example belongs to a single, specific fold

$$f_i = \{e_1, \dots, e_{|f_i|}\}$$

Model evaluation

Methods to estimate a model's performance – k -fold cross-validation

- The k -fold cross-validation method set-up



Model evaluation

Methods to estimate a model's performance – k -fold cross-validation

- The model undergoes training and evaluation in k iterations
- During each iteration, a specific fold (f_i) is designated as the **test set**, while the remaining $k - 1$ folds are assembled to form the **training set**

$$Tr_i = D \setminus f_i$$

$$Ts_i = f_i$$

- This ensures that each example participates in the training process an equal number of times and is used for testing exactly once
- Due to its relatively low bias and variance, **10-fold cross-validation** is generally recommended

Model evaluation

Methods to estimate a model's performance – k -fold cross-validation

- Proposed variants of the **k -fold cross-validation** method
 - **Leave-One-Out Cross-Validation**, in which k is set to the number of examples in the dataset
 - In each iteration, only one example is excluded for testing (the test set), while the remaining examples form the training set
 - **Stratified Cross-Validation** ensures balanced class distribution within each fold by creating stratified folds that mirror the class distribution of the entire dataset
 - **Repeated Cross-Validation** involves replicating the k -fold process multiple times, improving the reliability of the estimated generalization error
 - **Bootstrapping** uses random sampling with replacement to generate multiple subsets, offering a robust estimation of generalization error

Model evaluation

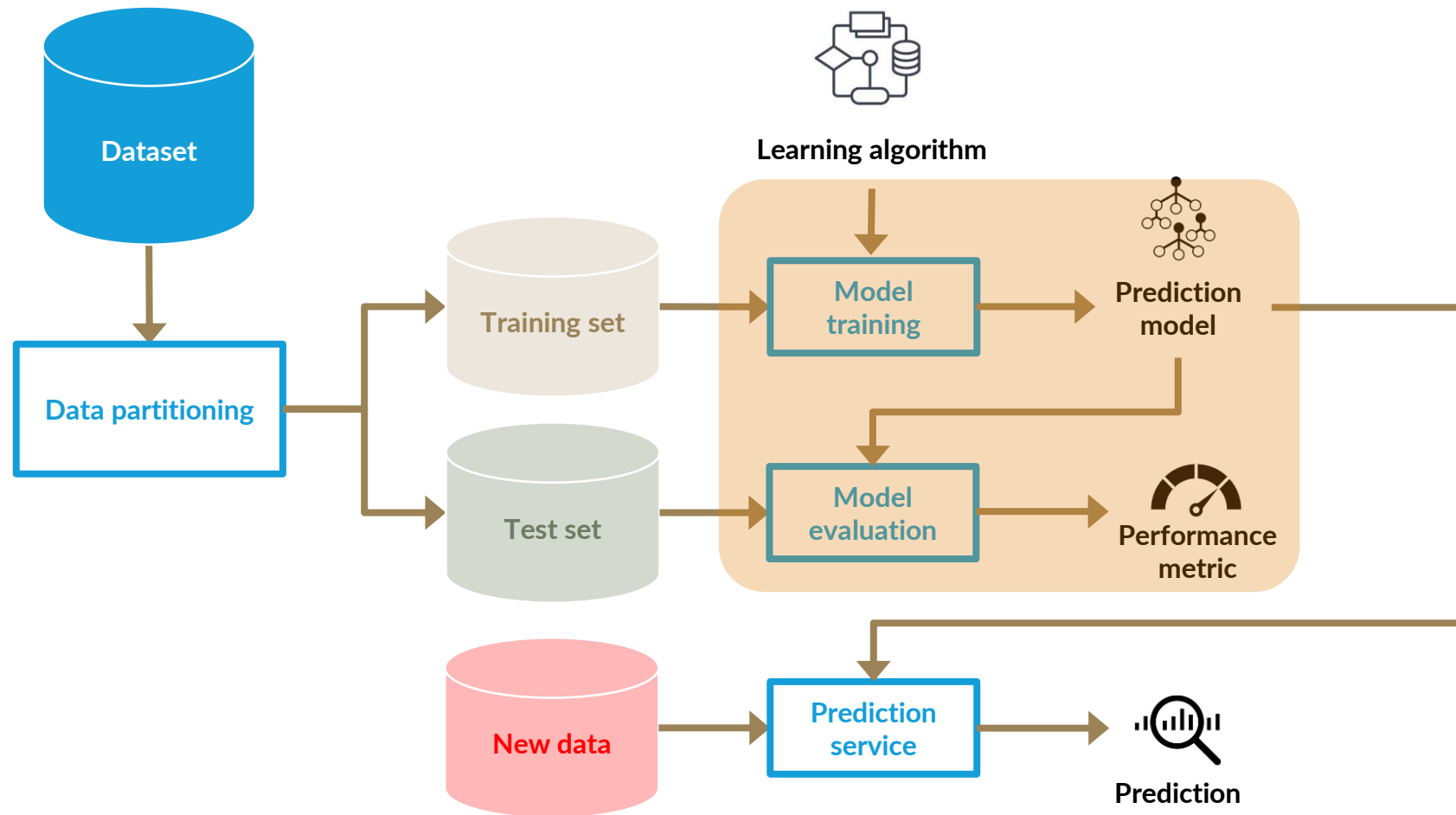
Methods to estimate a model's performance

Criterion	Holdout method	k -fold cross-validation method
Set-up	Split dataset into training and test sets once	Split dataset into k subsets; train k times, each time using a different fold as test set
Advantages	Simple and fast; easy to implement	More reliable estimate; uses all data for training; reduces variance; suitable for small datasets
Disadvantages	Performance estimate can vary depending on the split; less reliable for small or imbalanced datasets; does not use all data	More computationally expensive; repeated training k times; may be challenging for very large datasets or complex models
Recommended for	Large datasets, balanced classes, limited computational resources, simple models	Small datasets, imbalanced classes, need robust evaluation, moderate computational resources

Model evaluation

Methods to estimate a model's performance – Validation set

General experimental set-up



Model evaluation

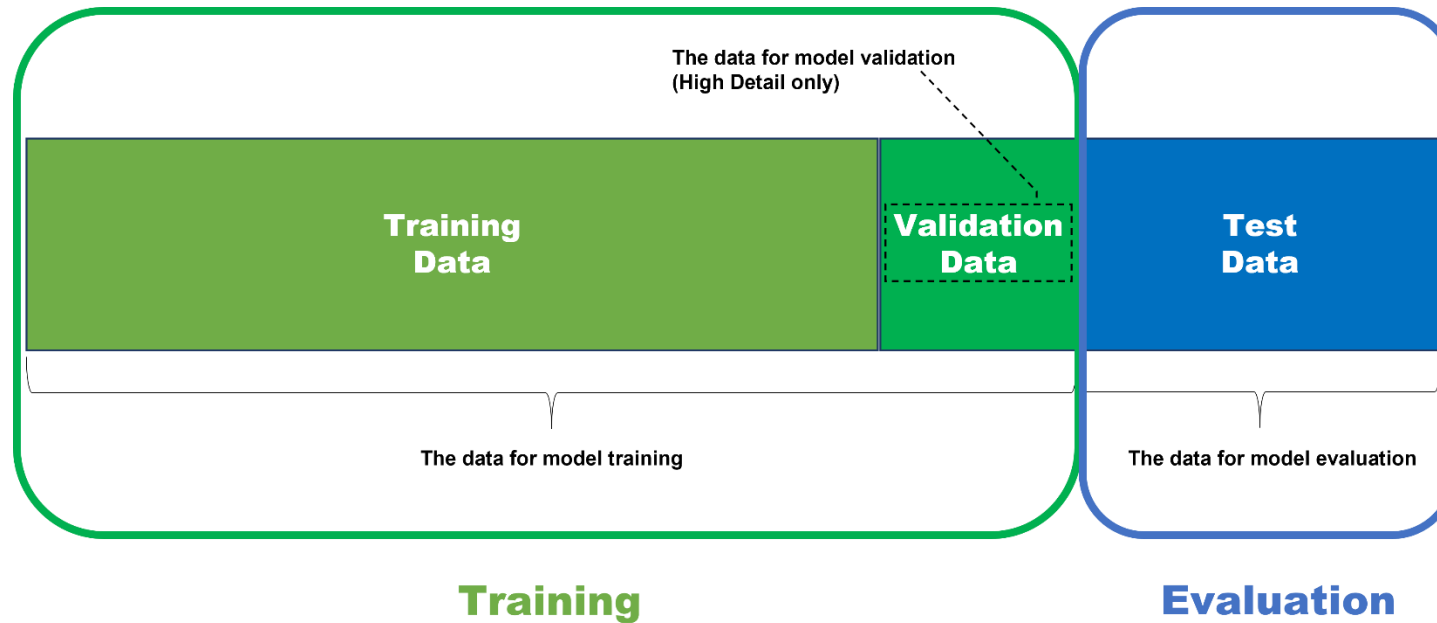
Methods to estimate a model's performance – Validation set

- To reliably tune and evaluate a model during development, a dedicated **validation set** is often used, which plays a key role in hyperparameter selection and performance assessment:
 - The **validation set** is a subset of the training data used during model development
 - Evaluates model performance independently of both training and test sets
 - Guides hyperparameter tuning (e.g., learning rate, regularization strength, tree depth)
 - **Hyperparameter** considerations:
 - Helps prevent overfitting by balancing bias and variance
 - Ensures the model does not simply memorize the training data
 - **Benefits:**
 - Supports informed model selection
 - Improves the reliability of the final performance estimate on the test set

Model evaluation

Methods to estimate a model's performance – Validation set

- **Validation set**

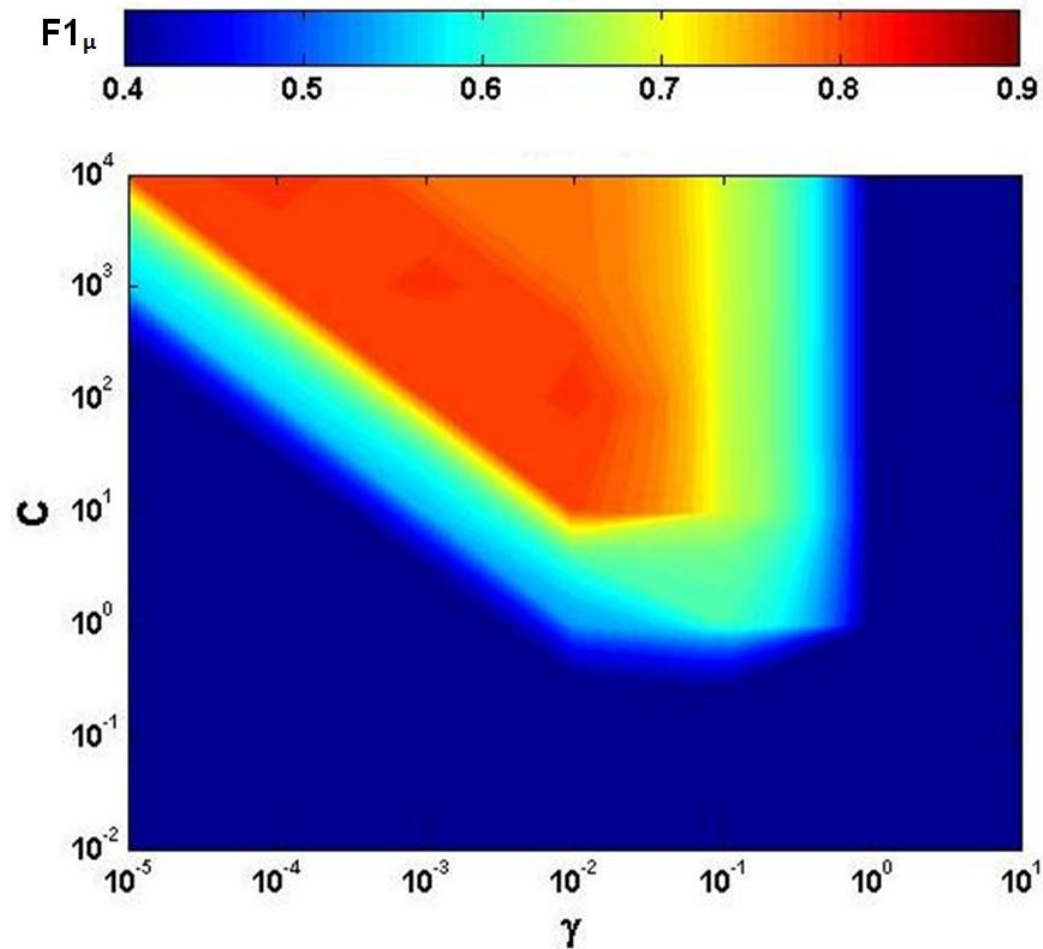


Prepare Training Set, Validation Set, and Test Set: https://support.cognex.com/docs/deep-learning_320/web/EN/deep-learning/Content/deep-learning-Topics/training-green-hd/prepare-train-valid-test.htm

Model evaluation

Methods to estimate a model's performance – Validation set

- Example of hyperparameter tuning of a RBF kernel SVM classifier



Parameters:

C – Regularization parameter

γ – Kernel coefficient for RBF

Measuring the Improvement of Web Page Classification
in Using Mark-up Features, Pedro Strecht, DSIE 2013

Model evaluation

Other evaluation criteria

- **Speed and scalability**

- Model construction time
- Model usage time
- Efficiency with disk-based databases

- **Robustness**

- To noise, missing values, irrelevant variables, and data flow

- **Interpretability**

- Understanding of the problem provided by the model

Model evaluation

Other evaluation criteria – Speed and scalability

- **Speed and scalability** – It is necessary to consider the time and space required in the two distinct phases of classification:
 - **Time to build (train) the model**
 - In the case of the simple linear classifier, it is the time needed to adjust the line
 - This time is proportional to the number of examples
 - **Time to use the model (make a prediction)**
 - In the case of the simple linear classifier, it is the time needed to test which side of the line an unclassified instance is on
 - This can be done using the same time (constant time)
 - Some classification algorithms are very efficient in one aspect and very poor in another

Model evaluation

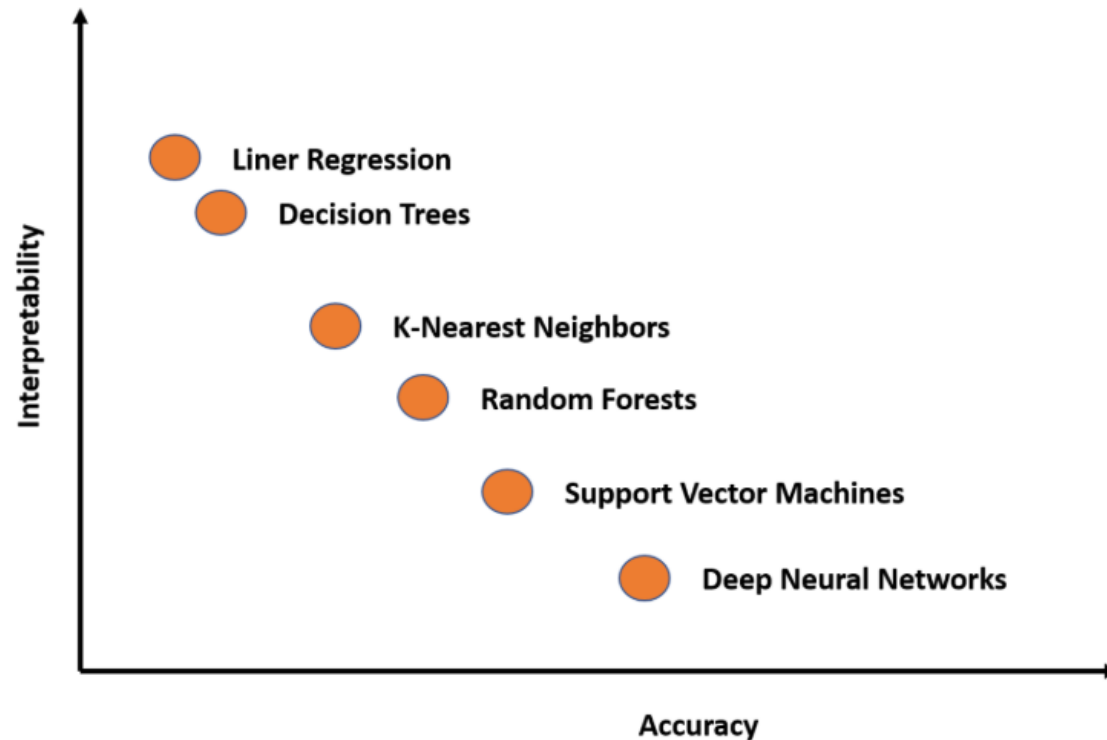
Other evaluation criteria – Robustness

- **Robustness** – It is necessary to consider what happens when there is:
 - **Noise**
 - For example, a person's age might be typed as 650 instead of 65
 - How does this affect the model?
 - This is only important in the construction of the model; if the example to be classified has noise, nothing can be done
 - **Missing values**
 - If we want to classify an insect, but we only know the length of the abdomen (X-axis), and we don't know the length of the antenna (Y-axis)
 - Can we still classify the example?

Model evaluation

Other evaluation criteria – Interpretability

- **Interpretability** – The structure of the classifier allows humans to obtain information about the topic being studied
 - Well-known trade-off between **interpretability** and **predictive performance**



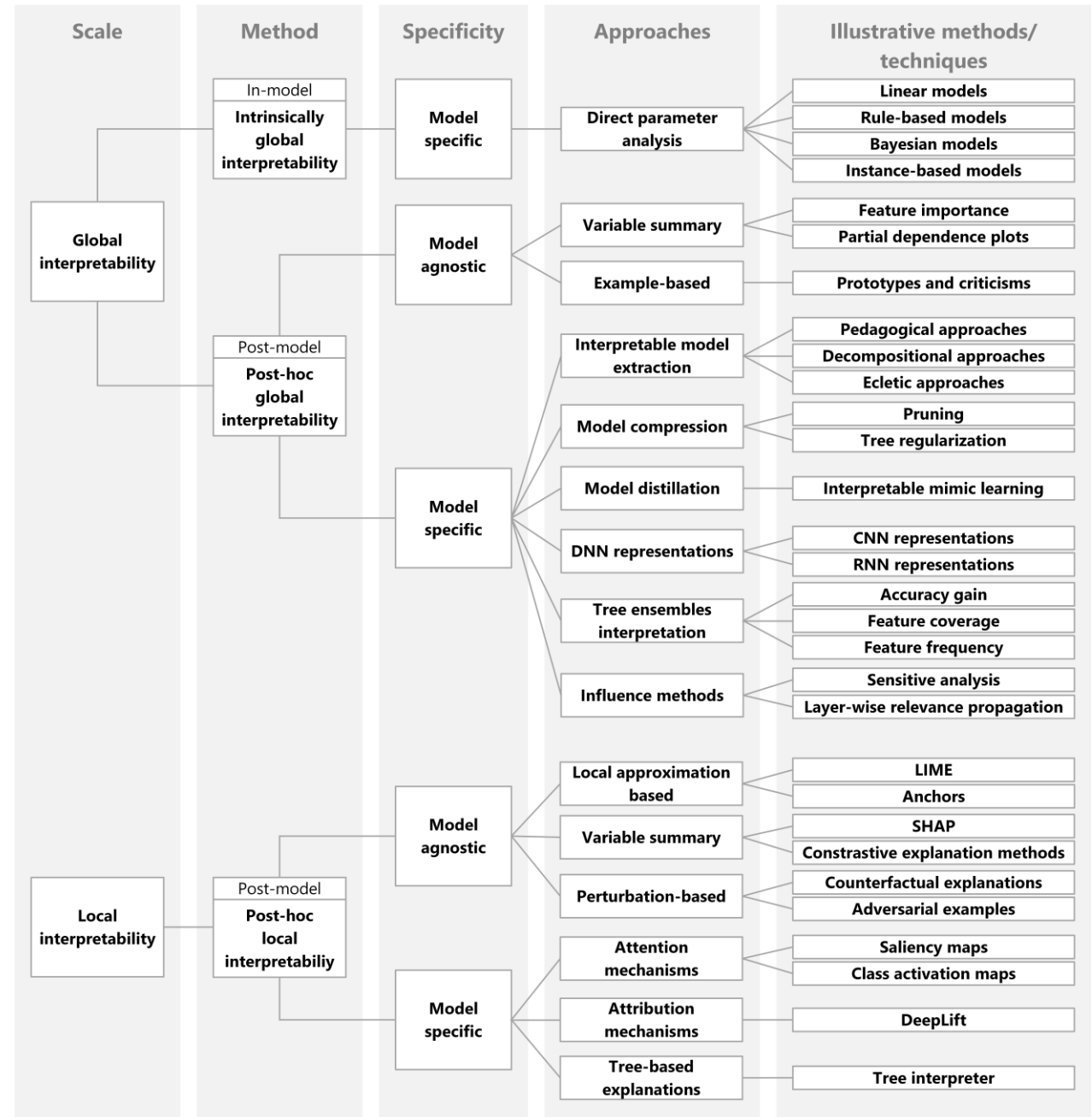
Decoding the Black Box: An Important Introduction to Interpretable Machine Learning Models in Python

<https://www.analyticsvidhya.com/blog/2019/08/decoding-black-box-step-by-step-guide-interpretable-machine-learning-models-python/>

Model evaluation

Other evaluation criteria – Interpretability

■ Interpretability taxonomy



Model evaluation

Case study 1 – Titanic Survival Prediction



- **Preprocessing:** Impute Age, handle missing Embarked; encode categoricals; normalize numeric features
- **Feature engineering:** FamilySize, IsAlone, Title (from Name), ticket group signals; bin Fare/Age if helpful
- **Model:** Logistic Regression (probability-based, discriminative) for interpretability or Decision Tree
- **Evaluation:** Stratified CV; metrics: Accuracy, ROC-AUC

Model evaluation

Case study 2 – House Prices



- **Preprocessing:** Target log-transform; impute domain-aware missing values; one-hot/ordinal encode; fix skewed vars
- **Model:** Gradient Boosting (e.g., XGBoost/LightGBM), optimization-based; strong on tabular with non-linearities
- **Evaluation:** K-fold CV with RMSE on log price; compare to linear baselines (Ridge/Lasso) for interpretability trade-offs

Model evaluation

Case study 3 – Telco Customer Churn



- **Preprocessing:** Clean TotalCharges; encode categoricals; treat imbalance with class weights or SMOTE
- **Model:** Decision Tree (search-based) or C5.0 for explicit, human-readable rules for operations teams
- **Evaluation:** Prioritize Recall/ROC-AUC and cost-aware metrics; calibrate thresholds by retention budget

Model evaluation

Case study 4 – Credit Card Fraud



- **Preprocessing:** Scale Amount/Time; stratified partitions; consider undersampling/oversampling; tune decision threshold
- **Model:** Support Vector Machine (optimization-based, margin) or Gradient Boosting for strong separation
- **Evaluation:** Precision-Recall curves and PR-AUC (recommended); monitor recall at low FPR

Model evaluation

Case study 5 – CIFAR-10 Object Recognition in images



- **Preprocessing:** Normalize pixels; strong data augmentation (flips/crops/color jitter); early stopping
- **Model:** Convolutional Neural Network (optimization-based, parametrized) for spatial hierarchies
- **Evaluation:** Top-1 accuracy; confusion matrix by class; monitor generalization gap